

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Software Engineering 2
Design Document

Author(s): **Francesco Rausa - 10833982**

David Ravelli - 10843295

Pietro Poggi - 10891051

Academic Year: 2025-2026
Version: 1.0
Release date: 07/01/2026

Contents

Contents **i**

1 Introduction **1**
1.1 Scope . . . 1
1.2 Definitions, Acronyms, Abbreviations . . . 2
1.3 Revision history . . . 3
1.4 Reference Documents . . . 3
1.5 Overview . . . 3
1.5.1 Document Structure . . . 3

2 Architectural Design **5**
2.1 Architectural Design . . . 5
2.1.1 Overview: High-Level Components and Interactions . . . 5
2.2 Component View . . . 7
2.2.1 Component Diagram . . . 8
2.2.2 Components Description . . . 8
2.3 Deployment View . . . 13
2.3.1 Detailed Layer Decisions . . . 14
2.4 Component Interfaces . . . 16
2.4.1 REST API Endpoints . . . 16
2.4.2 Other Interfaces . . . 19
2.5 Runtime View . . . 20
2.6 Selected Architectural Styles and Patterns . . . 46

3 User Interface Design **48**
3.1 General overview . . . 48
3.2 User Interface Layouts . . . 49
3.2.1 User Authentication . . . 49
3.2.2 Map and navigation . . . 50
3.2.3 Profile . . . 51
3.2.4 Trip history . . . 52
3.2.5 Automated trips . . . 53
3.2.6 Manual trip Insertion . . . 54

4 Requirements Traceability **55**

4.1 Functional Requirements Traceability . . . 55
4.2 Non-Functional Requirements Traceability . . . 61
4.2.1 Performance Requirements . . . 61
4.2.2 Software System Attributes . . . 62

5 Implementation, Integration and Test Plan 64
5.1 Overview . . . 64
5.2 Development Plan . . . 64
5.2.1 Phase 1: Infrastructure and Core Services . . . 64
5.2.2 Phase 2: Data Acquisition (The "Write" Side) . . . 65
5.2.3 Phase 3: Data Aggregation (The Core Business Logic) . . . 65
5.2.4 Phase 4: Navigation and Visualization (The "Read" Side) . . . 65
5.3 Integration and Test Strategy . . . 66
5.3.1 Unit Testing . . . 66
5.3.2 Integration Testing . . . 66
5.3.3 System and Acceptance Testing . . . 66

6 Effort Spent 68

7 References 69
7.1 Used Tools . . . 69
7.1.1 Use of ChatGPT . . . 69

List of Figures 71

List of Tables 72

1 Introduction

1.1. Scope

BikeBetterPath (BBP) is a platform that supports cyclists in discovering, evaluating, and using safe and comfortable routes in urban areas. The system allows registered users to record their bike trips, manually or automatically, and to submit contributions describing obstacles, roadworks, and other conditions observed on specific street segments.

The platform defines a Contribution as a structured report that associates one or more typed Information items with a StreetSegment, possibly linked to a Trip performed by a user. A StreetSegment is the atomic infrastructure unit in BBP, representing a portion of road between two consecutive intersections, with no intermediate junctions, so that each segment can be evaluated and updated independently. A BikePath is not a static object stored in the database, but a logical route dynamically computed by the system by chaining StreetSegments that are currently considered suitable for cycling, according to aggregated user feedback and automatically detected data.

The system automates key activities such as collecting sensor data from mobile devices, inferring potential obstacles, requesting user confirmation, and periodically aggregating contributions to maintain an up-to-date representation of street conditions. This aggregation follows a merge policy that weighs Information items based on their freshness, consistency, and number of independent confirmations, so that recent and widely supported reports have more influence on the segment's state. In particular, the system evaluates specialized BikePathSuitability Information to promote or demote StreetSegments as bike paths, highlights suitable segments on the map, and uses them as the basis for route computation. Additionally, it provides tools for users to visualize recorded trips, inspect current bike paths, and obtain personalized route recommendations that reflect their preferences and the latest crowd-sourced information.

The platform relies on external geographic data providers, such as OpenStreetMap, to support base map visualization and user navigation across the road network. These external datasets are used exclusively to provide the underlying cartographic representation and to assist in spatial operations (e.g., map matching).

The internal representation of bike paths and street conditions is instead fully managed by the system through a dedicated graph-based data model, which is incrementally built and updated from user-generated Contributions. This internal graph acts as an overlay over the base map and represents only those StreetSegments that have been identified, validated, and aggregated as relevant for cycling by the system.

1.2. Definitions, Acronyms, Abbreviations

- **System, Platform**: these terms are used interchangeably when referring to the system-to-be-developed (BBP).
- **BikePath**: a route suggested by the system that is composed only of StreetSegments currently marked as suitable for cycling.
- **StreetSegment**: an atomic portion of the road network between two intersections, which can be evaluated and updated independently. StreetSegments are identified through spatial matching against external map data and are internally represented and managed by the system.
- **Contribution**: a user-generated or automatically generated report containing one or more Information items about a StreetSegment.
- **Information**: an abstract fact attached to a Contribution (e.g., Obstacle, Roadwork, Warning, BikePathSuitability).
- **BikePathSuitability**: a specialized Information used to express and aggregate the suitability of a StreetSegment as part of a bike path.
- **Trip**: a cycling session performed by a registered user, either recorded automatically through sensors or manually specified.

Acronyms	Definition
RASD	Requirements Analysis & Specification Document
DD	Design Document
API	Application Programming Interface
REST	REpresentational State Transfer
DBMS	DataBase Management System
GPS	Global Positioning System
UI	User Interface
JSON	JavaScript Object Notation
HTTPS	HyperText Transfer Protocol over Secure Socket Layer
BBP	BikeBetterPath

Table 1.1: Acronyms used in the document.

Abbreviations	Definition
UC	Use Case
SD	Sequence Diagram

Table 1.2: Abbreviations used in the document.

1.3. Revision history

Revised on	Version	Description
19/12/2025	1.0	Initial Release of the document

Table 1.3: Revision history

1.4. Reference Documents

- [1] Di Nitto, Rossi, Camilli, *"A.Y. 2025-2026 Software Engineering 2 Requirement Engineering and Design Project"*, 2025.
- [2] Matteo Camilli, Elisabetta Di Nitto, Alfonso Fuggetta, Alessandro Margara, Matteo G. Rossi, Damian A. Tamburri, *Software Engineering: a Structured Journey from Inception to Delivery*, Bocconi University Press, 2025, ISBN: 9791280623508
- [3] *Microsoft Learn, Data Consistency primer*
- [4] *Chris Richardson, Microservices: API Gateway Pattern*
- [5] *Microsoft Learn, Data Partitioning guidance*
- [6] *Microsoft Learn, Relational vs. NoSQL data*
- [7] *OSM documentation*
- [8] *Overpass API*
- [9] *Microsoft Learn - Cloud-native data patterns*

1.5. Overview

1.5.1. Document Structure

This document is composed of seven sections:

- 1st Chapter - Introduction
- 2nd Chapter - Architectural Design
- 3rd Chapter - User Interface Design
- 4th Chapter - Requirements Traceability
- 5th Chapter - Implementation, Integration and Test Plan
- 6th Chapter - Effort Spent
- 7th Chapter - References

2 Architectural Design

2.1. Architectural Design

2.1.1. Overview: High-Level Components and Interactions

The architectural design of the **Best Bike Paths (BBP)** system is conceived to support a scalable, responsive, and data-intensive application. In order to handle heterogeneous data sources—such as real-time sensor streams, user-generated contributions, and external geospatial information—the system adopts a combination of well-established architectural paradigms.

The high-level architecture is characterized by the following principles:

- **Client–Server Architecture:** The system clearly separates the *Mobile Client*, responsible for user interaction, trip recording, and visualization, from the *Backend System*, which performs data processing, aggregation, and routing computation.
- **Four-Tier Architecture:** The system is logically organized into four tiers:
  1. **Presentation Tier:** Composed of the mobile application, which handles user interaction, map visualization, and sensor data acquisition (e.g., GPS, accelerometer, gyroscope).
  2. **Application Tier:** Includes the domain-specific services implementing the core functionalities of BBP, such as trip management, contribution handling, routing computation, and profile management. These services expose their functionalities through RESTful APIs.
  3. **Service Management Tier:** Hosts services that support the application tier, such as authentication and authorization, request routing, and integration with external services. External providers (e.g., map and weather services) are accessed exclusively to retrieve base data, while all application-specific semantics—such as bike path quality, obstacles, and routing scores—are computed and maintained internally.
  4. **Data Tier:** A polyglot persistence layer that manages long-term storage for heterogeneous data, including user profiles, raw trip data, processed contributions, and the weighted road network used for routing.
- **Service-Oriented / Microservice-Inspired Architecture:** Each major functionality of the system is implemented as an independent service, enabling loose coupling and selective scalability. For example, services responsible for contribution analysis and data

aggregation can be scaled independently from routing services to accommodate peaks in incoming trip data.

- **RESTful Communication:** Communication between the mobile client and backend services, as well as between backend services themselves, is performed through RESTful APIs over HTTPS.

High-Level Interaction and Data Flow

The BBP system follows a **data-driven routing pipeline**. The interaction begins when the Mobile Client submits raw trip data—either collected automatically through sensors or inserted manually—to the backend system.

The *Trip & Contribution Service* stores this information in the *Raw Data Database*, which is optimized for high-throughput write operations. Upon the creation of new contributions, relevant events are published to the *Message Broker*, enabling asynchronous processing.

These events are consumed by the *Data Aggregation Service*, which analyzes incoming data to detect obstacles, assess path conditions, and merge information from multiple users. The results of this processing are used to update the weighted representation of the road network stored in the *Routing Database*.

Finally, the *Map Service* computes and exposes optimal bike paths by combining the weighted routing graph stored in the *RoutingDBMS* with base geographic map data retrieved from an external map provider (e.g., OpenStreetMap). The internally maintained graph is overlaid on top of the external map and enriched with dynamically updated path quality scores derived from aggregated user contributions. By integrating these scores into the routing process, BBP is able to suggest routes that are not only efficient in terms of distance, but also safer and more comfortable for cyclists.

Note on Message Broker In this Design Document, Apache Kafka is taken as a reference technology for the message broker supporting the system's event-driven communication. This choice is motivated by Kafka's ability to provide asynchronous, scalable, and reliable data streaming, which fits well with the architectural requirements of BBP, such as decoupling between producers and consumers and support for high-throughput event processing. However, this design decision is not intended to be binding: any alternative message broker offering equivalent guarantees in terms of scalability, fault tolerance, ordering semantics, and asynchronous communication can be adopted without affecting the overall system architecture.

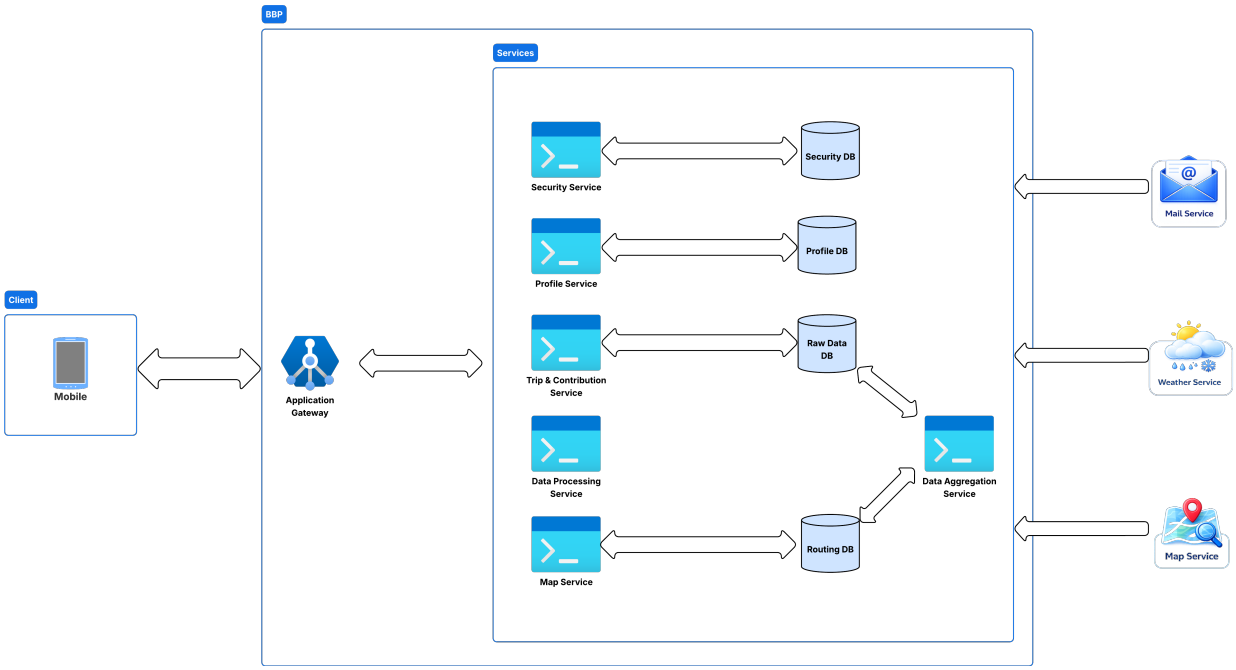


Figure 2.1: High-level overview of the BBP system architecture.

2.2. Component View

In this section, the main components of the BBP system are presented, together with a concise description of their responsibilities. Each component is derived from the product functions introduced in Section 2.2 of the RASD. Components addressing closely related functionalities are grouped together into higher-level services, in order to improve modularity, separation of concerns, and maintainability of the system architecture.

2.2.1. Component Diagram

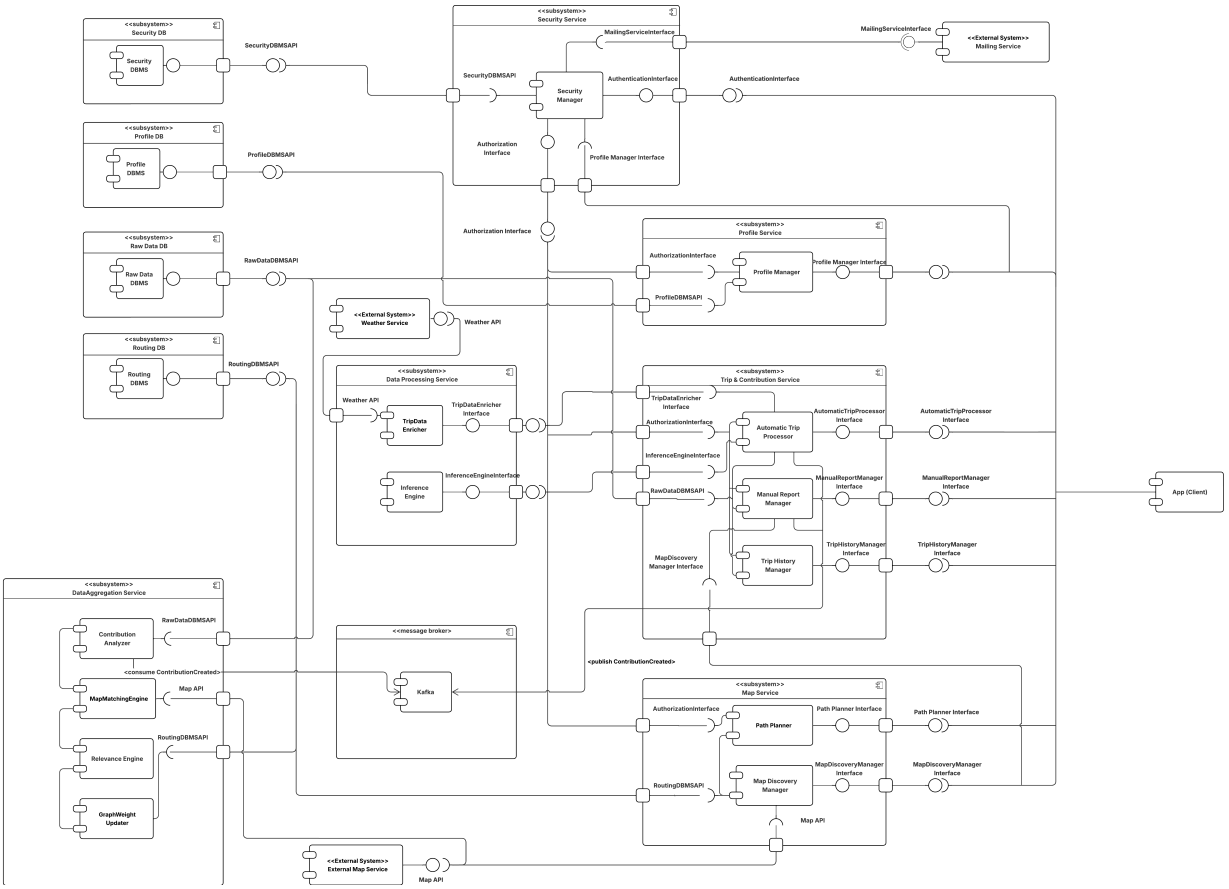


Figure 2.2: Component diagram of the BBP system.

2.2.2. Components Description

Each component is intended as a cohesive logical unit that encapsulates a set of collaborating internal elements to deliver the specified functionalities. The internal structure and implementation details of these elements are deliberately left unspecified, as they are not binding with respect to the overall system architecture, provided that each component satisfies the defined responsibilities. For the sake of clarity and readability, infrastructural elements such as the API Gateway are not represented explicitly in the component diagram, but are considered implicitly as part of the system's communication infrastructure.

1. **SecurityManager:** This component is responsible for authentication and authorization of users accessing the system. It manages login operations and token validation, ensuring that only authenticated users can invoke protected functionalities. The SecurityManager interacts with the SecurityDBMS, a relational DBMS storing credentials

and authentication-related information, and exposes authorization interfaces used by the other microservices.

2. **MailingService:** This component handles email-based communications required by the platform, such as account verification messages. It is invoked by the SecurityManager during registration procedures in order to decouple authentication logic from communication concerns.

3. **ProfileManager:** This component manages user profile information, including personal data and preferences that influence route ranking and visualization. It provides operations to retrieve and update profiles and persists such data through the ProfilesDBMS, a relational DBMS. All profile-related operations require prior authorization.

4. **AutomaticTripProcessor:** This component manages the lifecycle of automatically recorded trips. Trip recording is performed locally on the client, which collects GPS traces and sensor data until the user explicitly terminates the trip. When the user decides to save the recorded trip, the collected data is sent to the AutomaticTripProcessor, which invokes the Inference Engine to analyze sensor data and automatically generate preliminary contributions (e.g., road anomalies or quality indicators). The inferred data is then forwarded to the TripData Enricher, which computes advanced trip statistics and enriches the trip with contextual information such as weather conditions by interacting with an external Weather Service.

   The processed trip is returned to the AutomaticTripProcessor, which stores a draft copy in the RawDataDBMS, marked as non-definitive, and sends the enriched trip back to the client. At this stage, the user can review, modify, add, or remove contributions, including those generated automatically. Depending on user choice, the trip can be discarded, saved as private, or published. If the trip is published, the AutomaticTripProcessor stores the definitive copy marked as public and publishes a corresponding event to the Kafka message broker, enabling asynchronous aggregation and merging processes.

   It is worth noting that, at this stage, contributions are still associated with raw geographic coordinates. The association of each contribution to a specific street segment—considered the atomic working unit of the system—is performed later through a map matching process. This operation relies on external map data providers (e.g., OpenStreetMap) and is handled asynchronously by the Data Aggregation Service during the aggregation phase.

5. **ManualReportManager:** This component handles user-generated manual reports and manually declared trips. Users can submit individual reports or manually describe a completed trip through map interaction. During this process, the ManualReportManager

interacts with the Map Discovery Manager, which leverages an external map service to determine the geographic coordinates corresponding to the user's selection.

Manual submissions do not undergo inference or data enrichment and are stored directly in the RawDataDBMS using raw geographic coordinates. Depending on user choice, the data can be saved as private or published. If published, the ManualReportManager generates an event that is sent to the Kafka message broker.

Similarly to automatically generated contributions, the association of manual contributions to the system's atomic working unit (street segments) is performed asynchronously by the backend through a unified map matching process. This design choice allows the map matching logic to be implemented only once and shared across both manual and automatic contributions, ensuring consistency and scalability.

6. **TripHistoryManager:** This component provides functionalities to retrieve, list, and delete previously recorded trips. It accesses the RawDataDBMS to manage persisted trip data according to user requests.

7. **Inference Engine:** This component analyzes raw sensor data associated with automatically recorded trips in order to infer meaningful information, such as anomalies, obstacles, or road quality indicators.

8. **TripData Enricher:** This component enriches trip data with additional contextual and computed information. It integrates external data sources, such as weather conditions, by invoking an external Weather Service through REST calls. In addition, it computes advanced trip statistics (e.g., distance, duration, speed-related and performance metrics) based on the reconstructed path and collected raw data. The TripData Enricher is modeled as a separate component to allow future extensions with additional enrichment sources without impacting other parts of the system.

9. **ContributionAnalyzer:** This component consumes publication events from the Kafka message broker corresponding to newly published trips and manual reports. Rather than processing each contribution immediately, the ContributionAnalyzer applies internal aggregation policies—such as time-based windows or contribution-count thresholds—to determine when a sufficient and meaningful set of data is available.

   Once the defined conditions are met, the component initiates the aggregation workflow by selecting the relevant contributions from the RawDataDBMS and coordinating subsequent processing steps, including map matching, relevance evaluation, and updates to the global routing representation. This design enables scalable, batch-oriented processing while preventing unnecessary or premature updates to the system's global state.

10. **MapMatchingEngine:** This internal component of the Data Aggregation Service performs map matching operations by associating raw GPS traces and contribution coordinates to real-world street segments retrieved from the *External Map Service*. Map matching is executed exclusively during the aggregation phase and is not persisted in the RawDataDBMS, as its sole purpose is to support the update of the global routing graph.

11. **Relevance Engine:** This component evaluates the relevance of individual contributions by considering factors such as consistency, frequency, and recency, determining which contributions should influence the system's global state.

12. **GraphWeight Updater:** This component updates the weights and attributes of street segments stored in the RoutingDBMS, a graph-oriented DBMS. Updated weights reflect aggregated road conditions and bike-path suitability and are used to improve routing accuracy.

13. **Path Planner:** This component computes routes between two locations by executing routing algorithms over the graph stored in the RoutingDBMS. If no fully connected bike-path route exists, the Path Planner returns partial routes and coverage information, enabling graceful degradation.

14. **Map Discovery Manager:** This component provides map exploration functionalities. It retrieves base geographic map data from the*External Map Service* and overlays the internal bike-path graph derived from user contributions, allowing users to visually inspect available bike paths within a given geographical area.

15. **Kafka Message Broker:** Kafka is used as an asynchronous messaging infrastructure to decouple trip publication and manual reporting from aggregation and merging processes. It enables background processing of published data without blocking user interactions and supports scalable, event-driven communication between microservices.

16. **SecurityDBMS, ProfilesDBMS, RawDataDBMS, RoutingDBMS:** The SecurityDBMS and ProfilesDBMS are relational databases used to store authentication credentials and user profile data, respectively. The RawDataDBMS is a document-oriented database that stores trips and contributions in flexible JSON-like structures. The RoutingDBMS is a graph database that stores street segments and their relationships, supporting efficient routing and graph-based analyses. All DBMS components are modeled as passive elements accessed directly by the corresponding microservices.

17. **Weather Service (External System):** This external system provides meteorological data used during the data enrichment process. It is accessed through standard REST calls and is not part of the core system architecture.

18. **External Map Service (External System):** This external system provides base geographic map data and road network information used by the BBP platform for visualization, coordinate resolution, and map matching operations. The system does not impose constraints on the specific map provider to be adopted; any compliant service exposing geographic and road network data through standard APIs can be integrated. OpenStreetMap is a suitable candidate due to its open nature and wide availability, but alternative providers may be employed without affecting the system architecture.

19. **Mailing Service (External System):** This external service is responsible for handling email-based communications required by the platform. It is used, for instance, to send account registration and verification emails containing time-limited confirmation links. The Mailing Service is invoked by the Security-related components and is decoupled from the core system logic, allowing different email delivery solutions to be adopted without impacting the overall architecture.

2.3. Deployment View

The system is deployed as a distributed service architecture to ensure high scalability, resilience, and modularity. Each layer is distributed across a collection of servers and has distinct responsibilities, as described below:

- **Presentation Layer:** Represents the client in a traditional client–server architecture. Users interact with the BBP system through a mobile application, which is responsible for request initiation and data visualization.
- **Application Layer:** Hosts the services implementing the core business functionalities of the system. The deployment diagram illustrates a possible allocation of these services across different servers. The actual number of service instances and their distribution at runtime are dynamically determined by the orchestration platform, ensuring optimal resource utilization and resilience to varying traffic loads.
- **Service Management Layer:** Includes external and auxiliary services required by the BBP application. This layer comprises infrastructure services supporting the Data Layer, such as a replication and consistency service, as well as external services used to enrich application functionalities, such as map services or weather services providing route-related information.
- **Data Layer:** Handles persistent data storage using replicated database systems to guarantee fault tolerance and availability. The load balancer in the Service Management Layer ensures that requests are evenly distributed among database replicas.

Communication between layers occurs through REST APIs over HTTP/HTTPS, while interactions with the Data Layer rely on TCP/IP-based communication protocols.

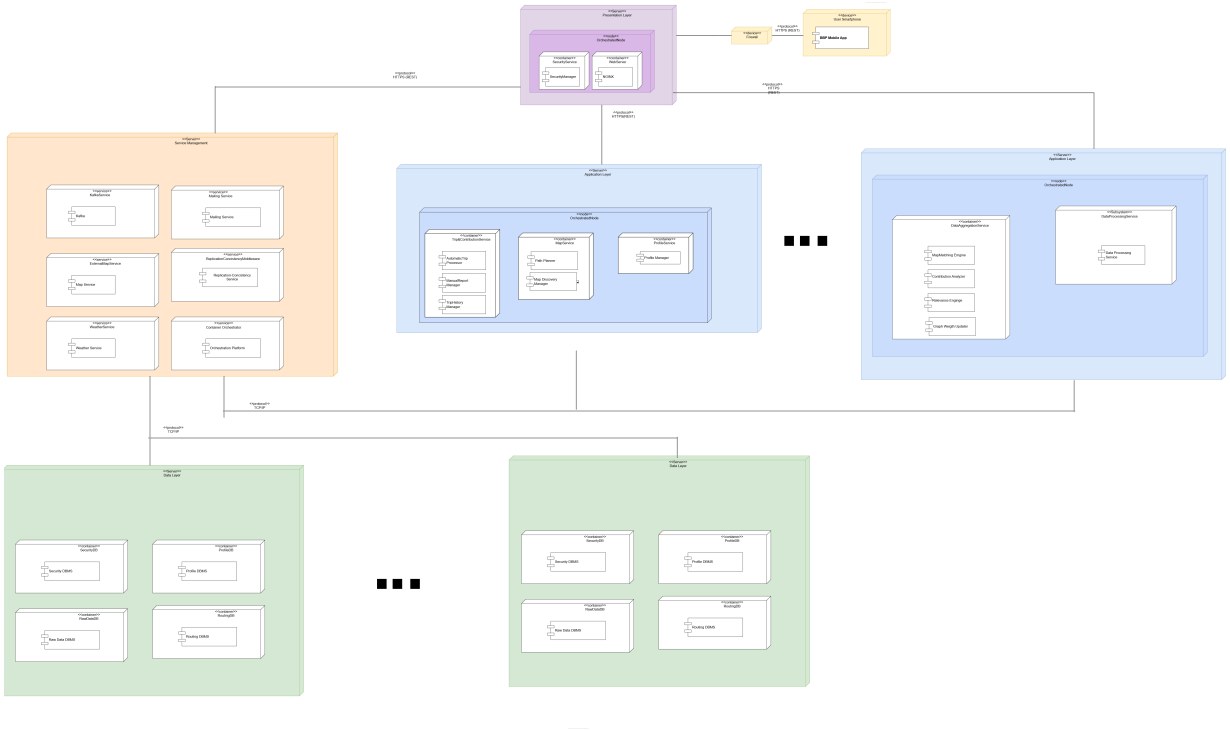


Figure 2.3: UML Deployment Diagram

2.3.1. Detailed Layer Decisions

The deployment architecture of the BBP system is designed to ensure high availability, scalability, and security through a containerized and distributed approach. The specific decisions regarding the layers are detailed below:

- **Web Server and API Gateway:** For clarity and performance, we selected **NGINX** as the web server acting as a reverse proxy and API Gateway. Deployed in a dedicated container, it serves as the single entry point for all client requests, hosting the API endpoints exposed to the mobile application. NGINX is responsible for forwarding requests to the appropriate microservices in the Application Layer. Additionally, it handles SSL/HTTPS termination, request filtering, and load balancing, distributing incoming traffic efficiently to prevent service overload and ensuring secure communication between the external world and the internal network. NGINX can be easily substituted by several separate dedicated components that each handle one of the functionalities.
- **Scalability and Containerization:** The architecture follows a microservices inspired pattern where each main component runs in an isolated container. This isolation allows for independent horizontal scaling: the Orchestrator platform monitors resource consumption and incoming traffic patterns, dynamically adjusting the number of replicas for each

service. For instance, during high-traffic events, the *Trip Service* can be scaled up independently of the *Profile Service*. In the Data Layer, replication is employed to improve read scalability, allowing the system to handle a high volume of concurrent read queries by distributing them across multiple database replicas.

- **Resiliency and Fault Tolerance:** The container orchestration platform plays a crucial role in system reliability by managing failover mechanisms. If a service container crashes or becomes unresponsive, the orchestrator automatically detects the failure and restarts the instance or redirects traffic to healthy replicas. Furthermore, the decoupling of services via the **Kafka Message Broker** ensures that temporary unavailability of the *Data Aggregation Service* does not block the ingestion of new trips, as events are queued and processed once the service is restored.
- **Orchestration Platform Considerations:** While a specific technology is not strictly binding, a platform like **Kubernetes** is the reference choice for managing the container lifecycle. The orchestrator provides essential features such as server-side service discovery, allowing NGINX and internal services to locate each other dynamically without hard-coded IP addresses. It also manages internal networking and secret management, simplifying the operational overhead of maintaining a distributed infrastructure.
- **Polyglot Persistence and Data Consistency:** The Data Layer is designed as a polyglot persistence environment, utilizing different DBMS technologies optimized for specific data types (Relational for profiles, Document for raw trips, Graph for routing). To ensure data availability, a Multi-Leader or Master-Slave replication strategy is adopted depending on the specific DBMS. This ensures that even in the event of a node failure, data remains accessible. Consistency is managed according to the data criticality: strict consistency is enforced for authentication and profiles (SecurityDBMS), while eventual consistency is accepted for the aggregation of crowd-sourced contributions into the routing graph.

2.4. Component Interfaces

In this section the component interfaces defined for the system are described, highlighting for each one the expected inputs and outputs.

2.4.1. REST API Endpoints

Each microservice of the system exposes one or more interfaces, corresponding to the resource endpoints of the defined REST API. All protected endpoints require prior authorization through the `/authorization/validateToken` interface, which is implicitly invoked by the API Gateway before forwarding requests to the target microservice.

All API endpoints must be accessed through the HTTPS POST method, unless explicitly stated otherwise. Input parameters and output values are provided in JSON format within the request and response bodies.

To improve readability, only user-driven API calls are documented in this section.

The general structure of an API request is the following:

`https://DOMAIN/api/VERSION/INTERFACE/FUNCTION`

/authentication

- `/login`
  parameters: { username: String, passwordHash: String }
  responses:
  - **200 OK:** body: { accessToken: String, refreshToken: String }
  - **400 Bad Request:** body: { error: Enum(wrongCredentials) }
- `/refreshToken`
  parameters: { refreshToken: String }
  responses:
  - **200 OK:** body: { accessToken: String }
- `/logout`
  parameters: { accessToken: String }
  responses:
  - **204 No Content**

/authorization

- `/validateToken`
  parameters: { token: String }
  responses:
  - **200 OK:** body: { authorized: true }
  - **401 Unauthorized:** body: { authorized: false }

/profiles

- `/getPreferences`
  parameters: { userId: String }
  responses:
  - **200 OK:** body: { preferences: MapPreferences }
- `/updatePreferences`
  parameters: { preferences: MapPreferences }
  responses:
  - **200 OK**
- `/deleteAccount`
  parameters: { userId: String, passwordHash: String }
  responses:
  - **204 No Content**
  - **400 Bad Request:** body: { error: Enum(wrongCredentials) }

/map

- `/getOverlay`
  parameters: { boundingBox: BoundingBox, preferences: MapPreferences (optional) }
  responses:
  - **200 OK:** body: { geoJson: GeoJSON }
- `/getSegments`
  parameters: { boundingBox: BoundingBox }
  responses:
  - **200 OK:** body: { segments: [StreetSegment] }

- `/getSegmentDetails`
  parameters: { segmentId: String }
  responses:
  - **200 OK:** body: { segment: StreetSegment, obstacles: [Contribution] }
- `/getObstacleDetails`
  parameters: { contributionId: String }
  responses:
  - **200 OK:** body: { contribution: Contribution }
- `/calculateRoute`
  parameters: { start: Coordinate, end: Coordinate, preferences: MapPreferences (optional) }
  responses:
  - **200 OK:** body: { route: Route }

[1]

/trips/automatic

- `/submit`
  parameters: { tripData: RawTripData }
  responses:
  - **200 OK:** body: { enrichedTrip: TripPreview }
- `/save`
  parameters: { tripId: String, isPublic: Boolean }
  responses:
  - **200 OK**
- `/discard`
  parameters: { tripId: String }
  responses:
  - **204 No Content**

[1] Due to the incremental and contribution-driven nature of bike path data, the PathFinder component is designed to never fail. When a fully connected route cannot be computed, the system returns the best available partial route together with metadata describing the uncovered portion, ensuring graceful degradation and preserving usability.

/reports/manual

- `/submit`
  parameters: { reportData: ManualReport, isTrip: Boolean, isPublic: Boolean }
  responses:
  - **200 OK**

/trips/history

- `/list`
  parameters: { userId: String }
  responses:
  - **200 OK:** body: { trips: [TripSummary] }
- `/getDetails`
  parameters: { tripId: String }
  responses:
  - **200 OK:** body: { trip: Trip }

2.4.2. Other Interfaces

Kafka The Kafka message broker is used exclusively for asynchronous, event-driven communication between backend services and does not expose REST interfaces. Internal communication between components belonging to the same microservice is not exposed and therefore omitted.

External Geographic Data Provider The system relies on an external geographic data provider to obtain base map information and road network data used for visualization and map matching operations. The architecture does not impose constraints on the specific provider to be adopted, as long as it exposes suitable geographic and topological information through standard APIs.

For instance, if OpenStreetMap were used as the external provider, its Overpass API could be leveraged to query low-level map elements such as nodes, ways, and relations. Through such APIs—based on a C-like query language—it is possible to associate raw geographic coordinates with corresponding road segments, thereby supporting backend map matching operations. These interactions are entirely internal to the backend components and are not exposed as REST APIs.

2.5. Runtime View

In this section, the dynamic behavior of the system is described. For each Use Case identified in Section 3.1 *Use Cases and Activity Diagrams* of the RASD, the corresponding Sequence Diagram is provided, highlighting the interactions and messages exchanged between the involved components.

As in previous sections, irrelevant details are omitted for clarity. In particular, most client-side interactions between the User and the Client App (e.g., form filling or minor UI interactions), implicit authorization checks performed by the API Gateway, and extremely frequent operations such as calls to the `/authorization/validateToken` endpoint are not explicitly shown, as they would unnecessarily increase the complexity of the diagrams.

SD1. Sign Up

The User initiates the sign-up process by clicking the "Sign Up" button on the homepage of the Client App and submitting the required registration data through the `/api/v1/authentication/login` interface. The request is forwarded by the API Gateway to the `SecurityManager`, which validates the completeness and correctness of the submitted information, including password security requirements.

The `SecurityManager` then queries the `SecurityDBMS` to verify whether the provided email address is already associated with an existing account. If the email is already registered, the sign-up process is interrupted and the User is redirected to the login page. If the validation is successful, the `SecurityManager` creates a new, unverified user account in the `SecurityDBMS` and invokes the `MailingService` to send a verification email containing a confirmation link with a limited validity period of 24 hours.

When the User receives the verification email and clicks on the confirmation link before its expiration, a request is issued to the `/api/v1/authentication/verifyProfile` endpoint. The `SecurityManager` marks the account as verified in the `SecurityDBMS` and grants the User access to the profile completion page. At this point, the User can insert the remaining personal information required to complete the profile, after which the account becomes fully active and the User is allowed to access the platform functionalities.

If the verification link expires before being used, the `SecurityManager` invalidates the link and removes the previously stored unverified account data from the `SecurityDBMS`, forcing the User to repeat the registration process.

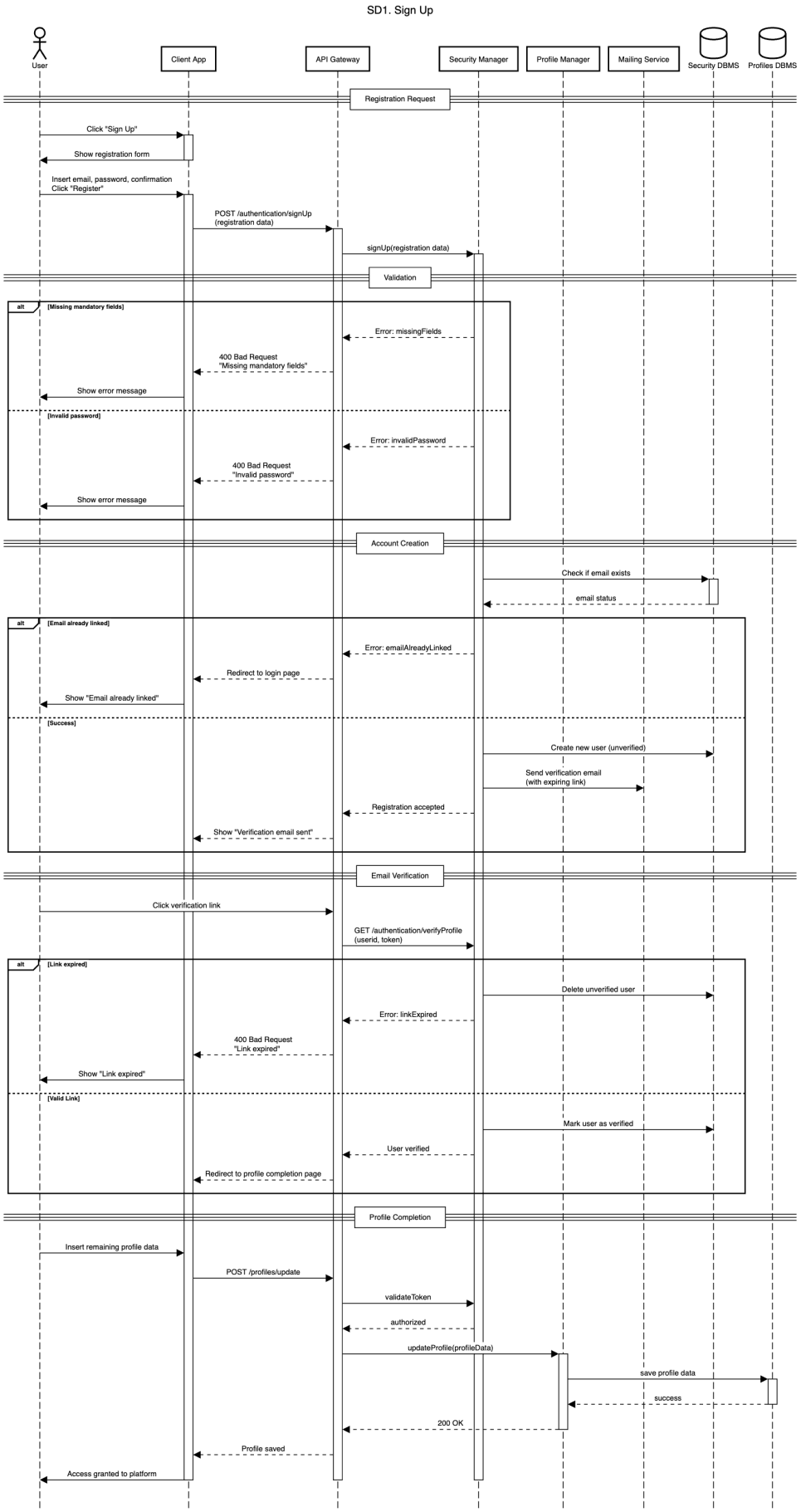


Figure 2.4: Sign Up

SD2. Sign In

The User initiates the sign-in process by clicking the "Sign In" button on the homepage of the Client App and submitting their credentials through the `/api/v1/authentication/login` API endpoint. The request is routed by the API Gateway to the `SecurityManager`, which validates the provided credentials by querying the `SecurityDBMS`.

If the submitted credentials are not valid, the authentication process is immediately interrupted and the User is notified that the entered credentials are incorrect. Similarly, if the account associated with the provided email address has not yet been verified, the `SecurityManager` denies access and informs the User that the account is not verified.

If the credentials are valid and the account is verified, the `SecurityManager` completes the authentication process by generating the access tokens and checking whether the User profile is complete. If the profile is complete, the User is redirected to the Home page of the platform. Otherwise, the User is redirected to the profile completion page, where the missing personal information can be provided.

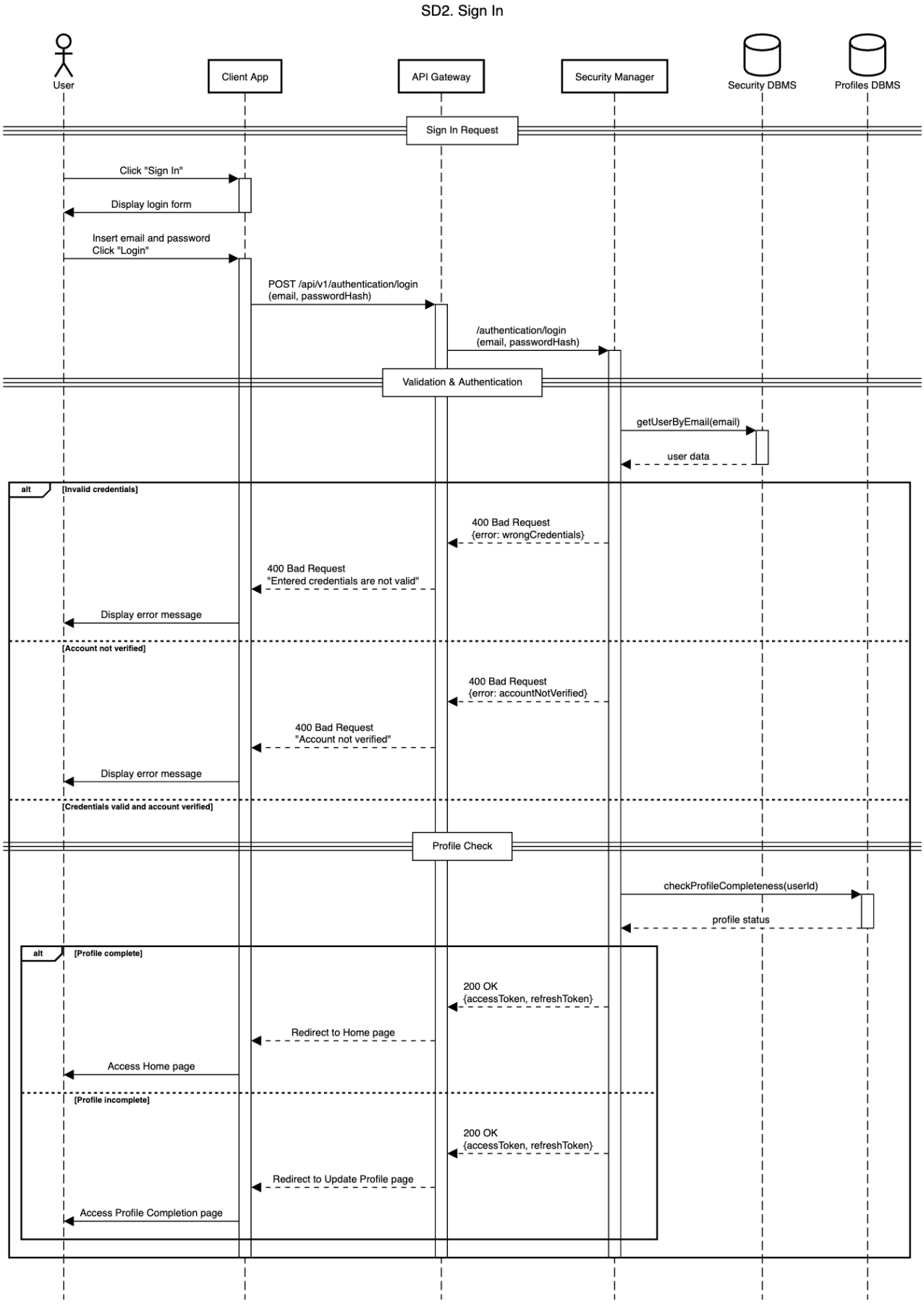


Figure 2.5: Sign In

SD3. Delete Account

The Registered User initiates the account deletion process by accessing the account settings page of the Client App and selecting the "Delete account" option. The system displays a warning explaining the consequences of account deletion and asks the User to confirm the operation. If the User cancels the process, no further action is performed and the User is returned to the account settings page.

If the User confirms the deletion, the system requires re-authentication by requesting the User's credentials. The credentials are submitted through the `/api/v1/profiles/deleteAccount` API endpoint and forwarded by the API Gateway to the `SecurityManager`. The `SecurityManager` validates the provided credentials by querying the `SecurityDBMS`. If the credentials are not valid, the deletion process is aborted and the User is notified accordingly.

If the credentials are valid, the `SecurityManager` orchestrates the account deletion process. The User profile and preferences are removed from the `ProfilesDBMS`, all trips associated with the User are deleted from the `RawDataDBMS`, and authentication credentials are removed from the `SecurityDBMS`. Additionally, all active sessions and authentication tokens associated with the User are invalidated to prevent further access.

Published contributions are not removed during this process, as they have already been aggregated and merged into the routing graph and can no longer be uniquely associated with a specific User.

Finally, the system confirms the successful deletion of the account to the User, who is no longer able to access the platform using the previous credentials.

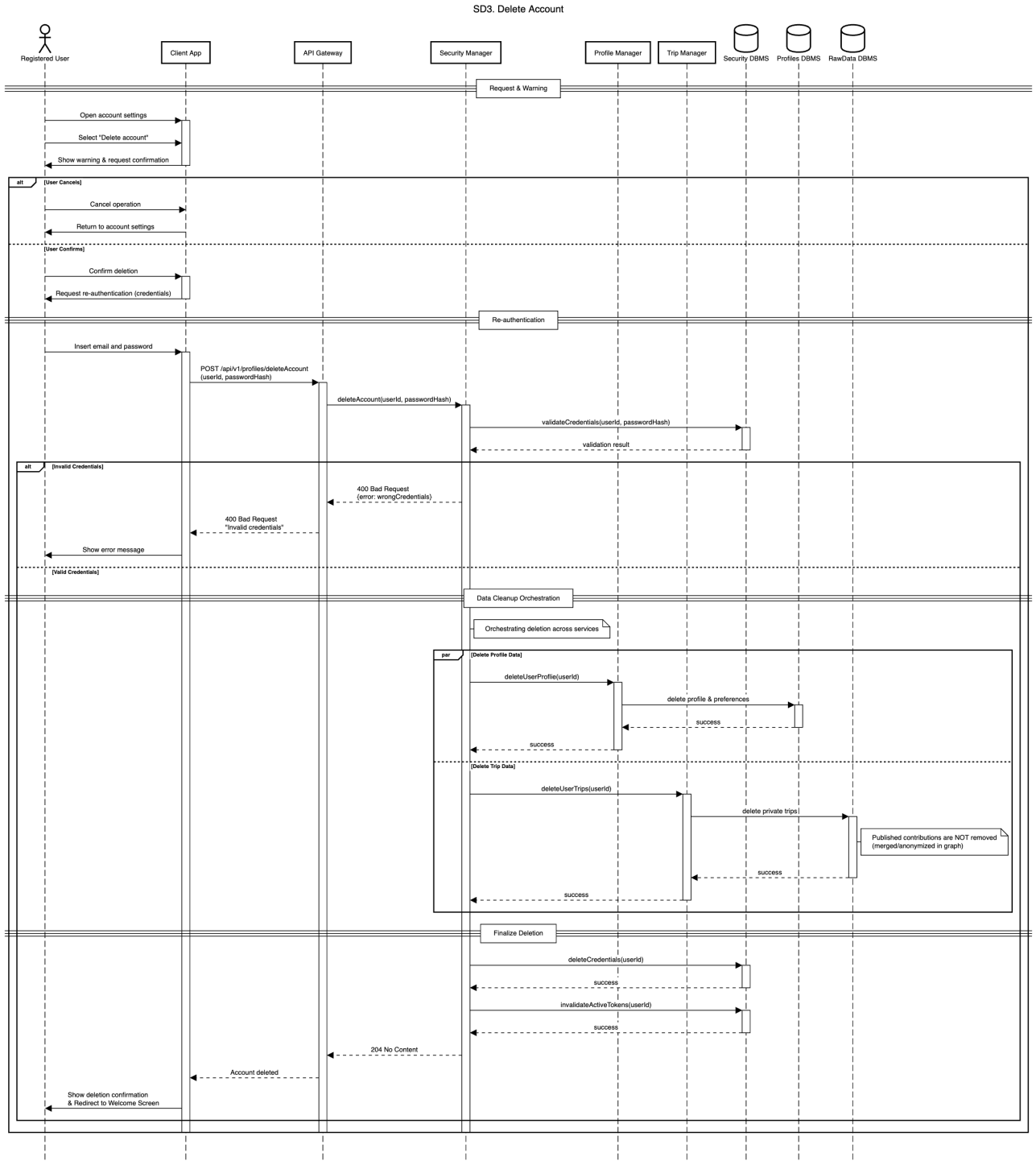


Figure 2.6: Delete Account

SD4. Logout

The User initiates the logout process by clicking the "Logout" button within the Client App. The logout request is sent through the `/api/v1/authentication/logout` API endpoint and forwarded by the API Gateway to the `SecurityManager`.

Upon receiving the request, the `SecurityManager` invalidates the authentication token associated with the User by updating the `SecurityDBMS`, effectively terminating the active session. Once the token has been invalidated, the system confirms the successful logout operation.

Finally, the User is redirected to the Home screen of the platform and is no longer authenticated, preventing access to protected functionalities until a new login is performed.

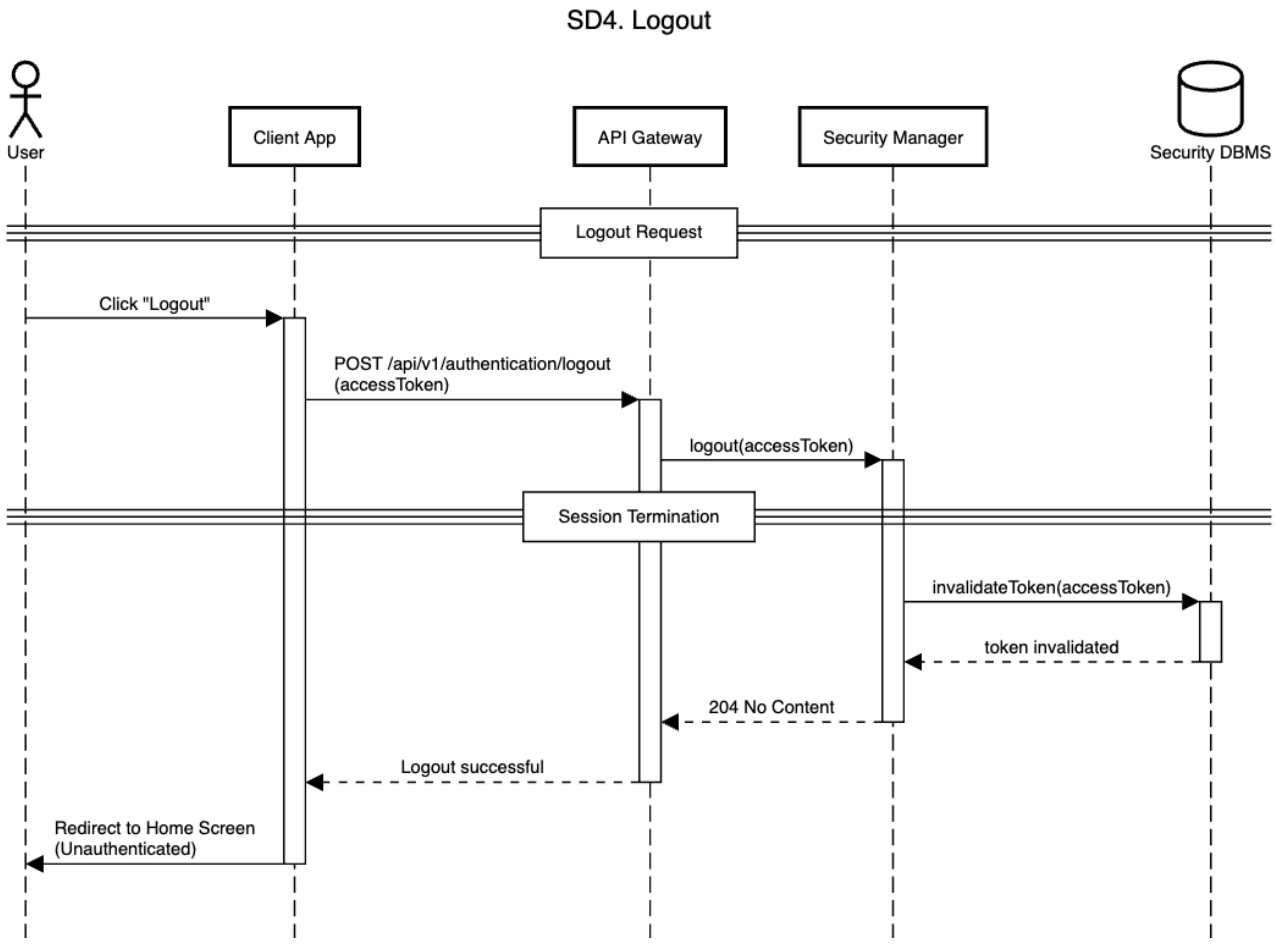


Figure 2.7: Logout

SD5. Edit Profile

The Registered User accesses the profile management section of the Client App by clicking on the "Profile" area and selecting the "Edit profile" option. The system retrieves the current profile information and visualization preferences by invoking the `/api/v1/profiles/getPreferences` API endpoint, which is forwarded by the API Gateway to the `ProfileManager`. The retrieved data is then displayed to the User in an editable form.

The User updates the desired personal information and preferences, such as path visualization filters derived from previous trips, and confirms the changes by clicking the "Apply" button. The updated data is submitted through the `/api/v1/profiles/updatePreferences` API endpoint and handled by the `ProfileManager`.

The `ProfileManager` validates the submitted changes, checking both data consistency and supported preference values. If the validation fails, the update process is aborted and the User is notified that the provided information is not valid. No modification is applied to the stored data.

If the submitted data is valid, the `ProfileManager` updates the User profile and preferences in the `ProfilesDBMS`. Upon successful completion of the operation, the system notifies the User that the profile has been successfully updated, ensuring that the stored information remains consistent across the platform.

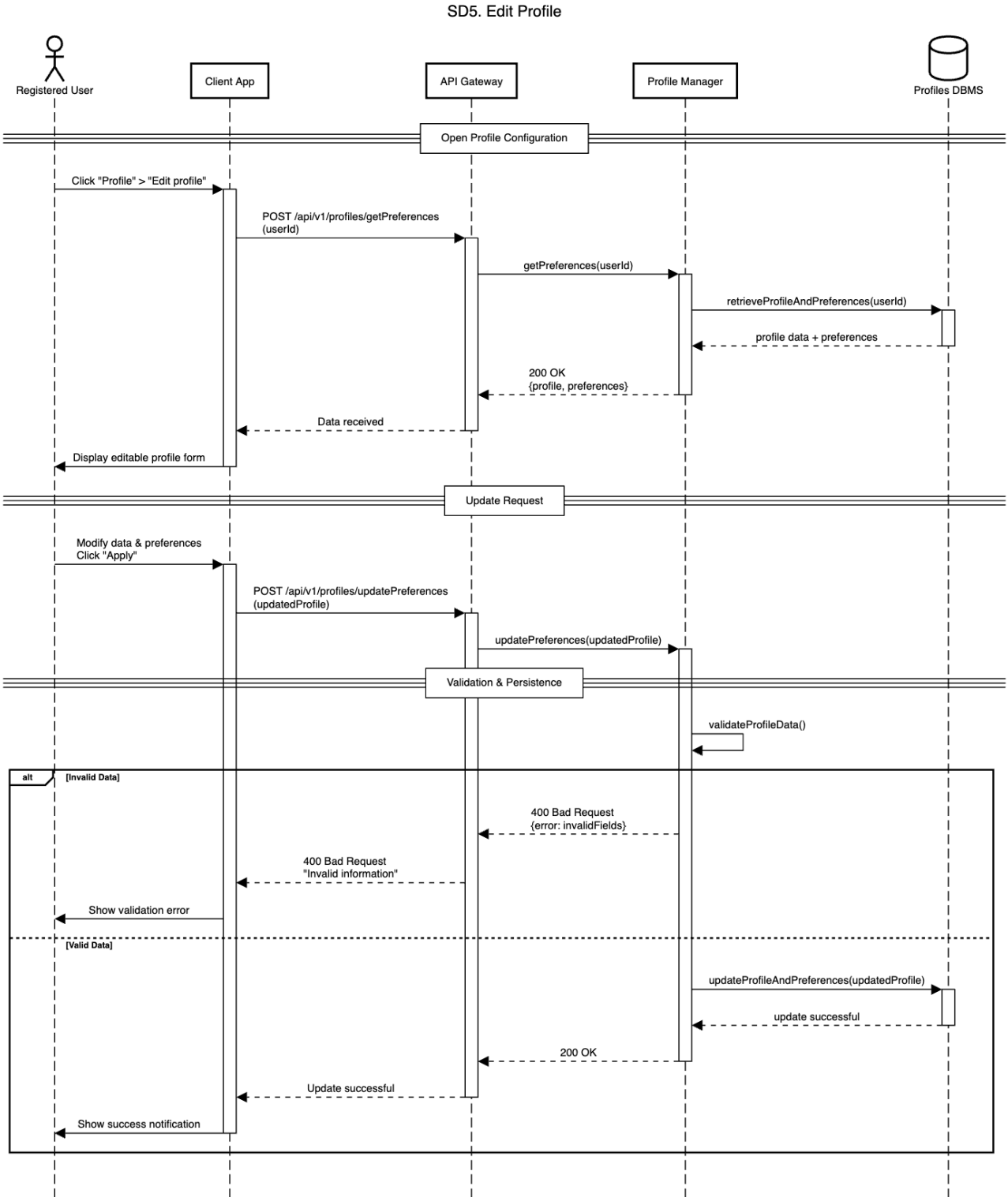


Figure 2.8: Edit Profile

SD6. View Trip History

The Registered User accesses the trip history section of the Client App by selecting the "My trips" option. The system retrieves the list of trips associated with the User by invoking the `/api/v1/trips/history/list` API endpoint, which is forwarded by the API Gateway to the `TripHistoryManager`. The `TripHistoryManager` queries the `RawDataDBMS` to retrieve all stored trips belonging to the User.

If no trips are available, the system displays an empty list together with an informative message. Otherwise, the list of available trips is shown to the User, who can select one of them to inspect its details.

When a trip is selected, the Client App invokes the `/api/v1/trips/history/getDetails` API endpoint to retrieve the full trip data, including its associated contributions. The retrieved information is displayed to the User.

If the selected trip has no associated contributions, the system simply shows the trip data without triggering any further action. If contributions are present, the User may choose to publish them. In this case, the system iteratively processes each contribution, publishing them one at a time. Each publication generates an asynchronous event that is forwarded to the Kafka message broker, allowing the Data Aggregation Service to incrementally merge the contribution data and update the routing graph according to the defined merging and update policies as detailed in the following sequence diagram *Publish Contribution*.

Once all selected contributions have been processed, control is returned to the trip view, allowing the User to continue browsing the application.

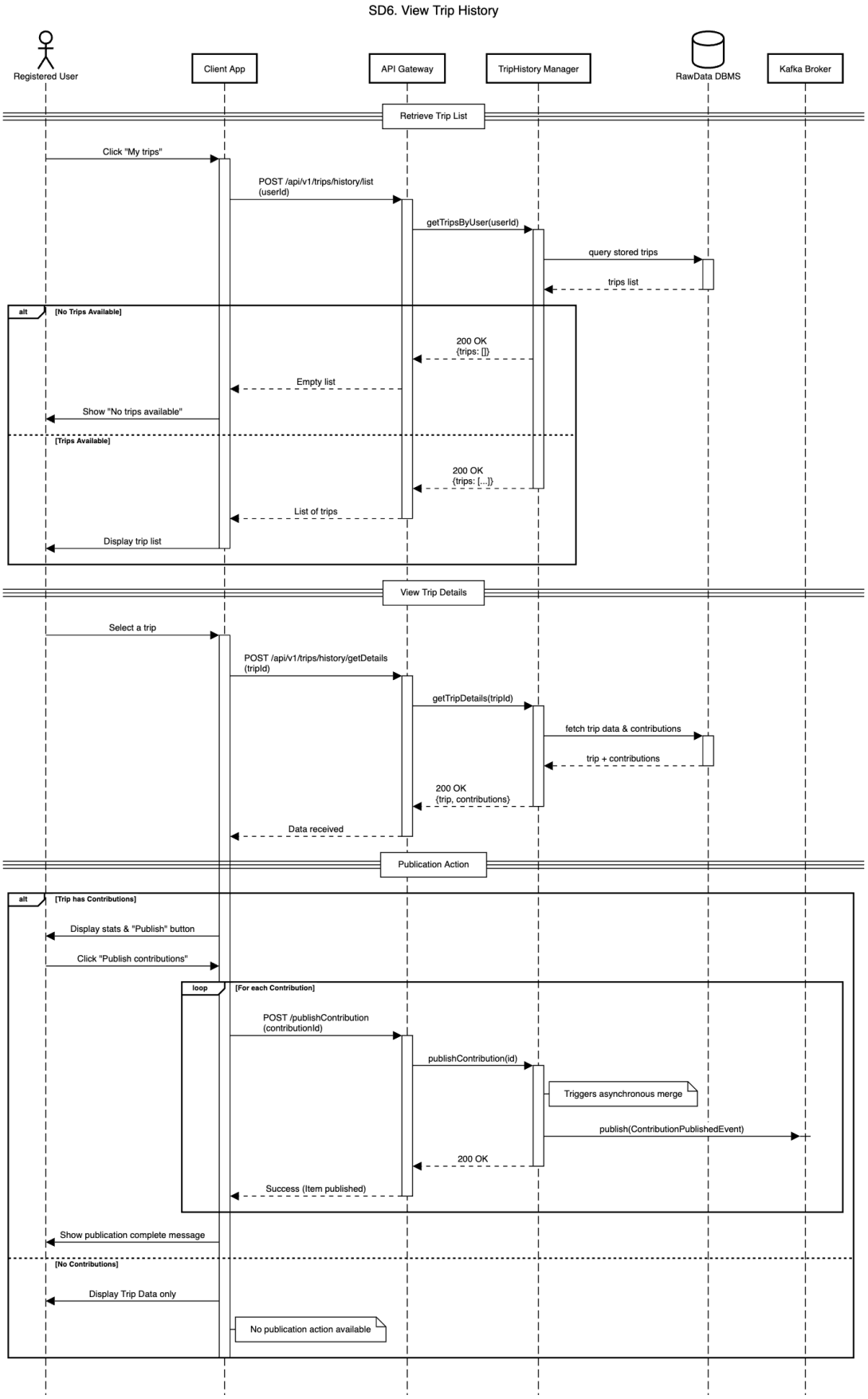


Figure 2.9: View Trip History

SD7. Publish Contribution

This use case describes the asynchronous processing and merging of a Contribution into the routing graph. It is not directly initiated by the User, but is triggered by other use cases (e.g., viewing the trip history or publishing an automatic or manual trip) when a Contribution is marked as public. Upon publication, the producing service emits a `ContributionPublishedEvent` to the Kafka message broker.

The ContributionAnalyzer periodically consumes events from Kafka and retrieves the corresponding Contribution data from the `RawDataDBMS`. The ContributionAnalyzer component first validates the Contribution, ensuring that all mandatory data is present and that the spatial and temporal information is consistent. If the Contribution is not valid, the process is aborted and the merge status of the Contribution is marked as `FAILED`, without modifying any existing path information.

If the Contribution is valid, the `ContributionAnalyzer` sends a request to the `MapMatchingEngine` which retrieves a list of Street segments form external map services, associates the contribution to the correct segment returning it to the `ContributionAnalyzer` who retrieves all existing Contributions referring to the same street or path segment from the `RawDataDBMS`. The `RelevanceEngine` then filters the retrieved data, discarding outliers and selecting only the most relevant Contributions according to predefined criteria such as severity, frequency, and recency.

The `GraphWeightUpdater` applies a weighted incremental aggregation policy to merge the new Contribution with the existing information and updates the corresponding segment weight in the `RoutingDBMS`. This approach allows the routing graph to be updated incrementally and efficiently, without reprocessing the entire dataset.

Finally, upon successful completion of the merge process, the Contribution is marked as `MERGED` in the `RawDataDBMS`. Publishing a Contribution and merging it into the routing graph are treated as two distinct concerns: publication is the result of an explicit User action, while merging is handled asynchronously by the system to ensure scalability, responsiveness, and eventual consistency of the routing data.

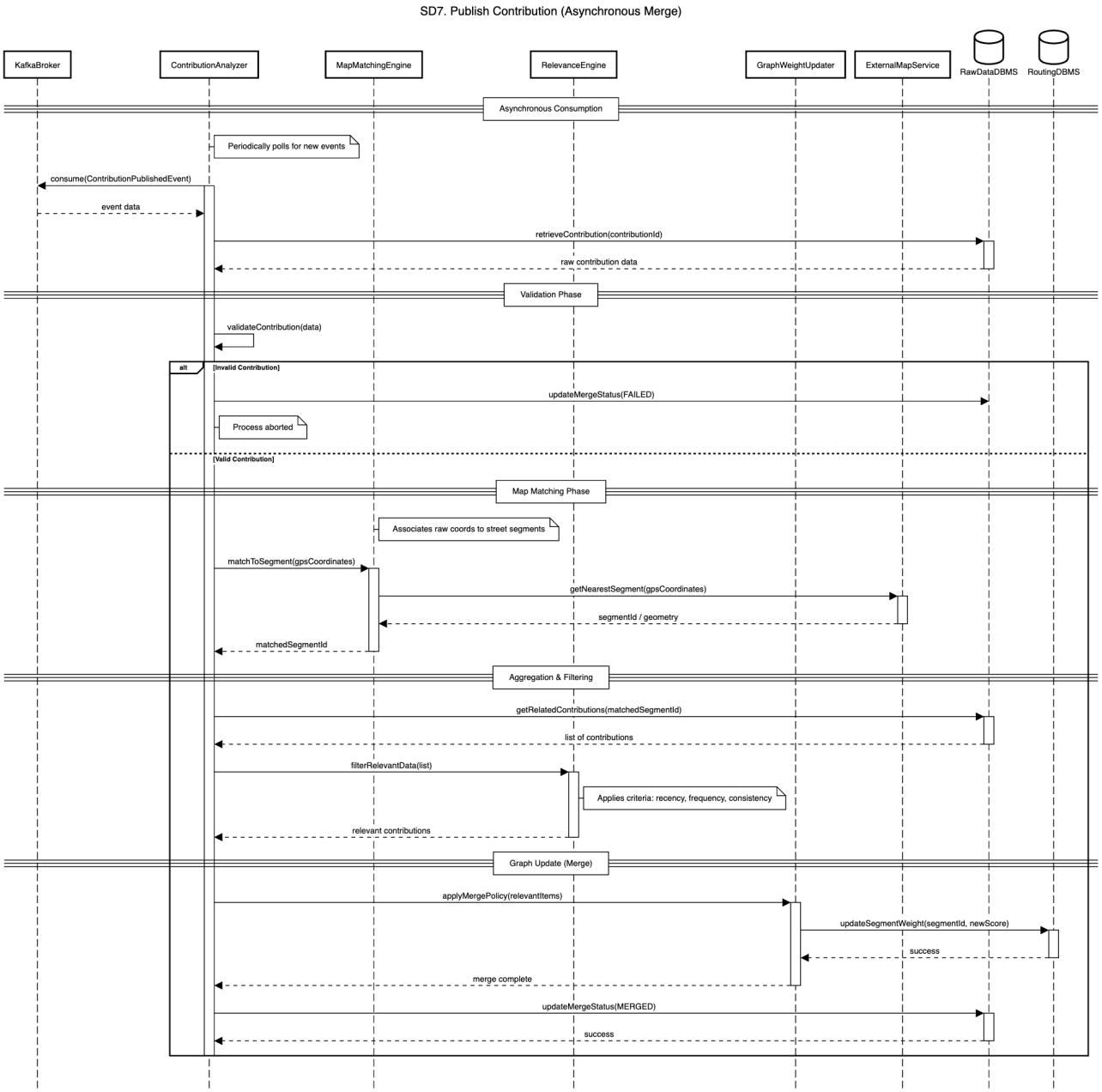


Figure 2.10: Publish Contribution

SD8. View Map

The Registered User or Unregistered User initiates this interaction by accessing the map section of the Client App or by panning and zooming to a new geographical area. The Client App calculates the bounding box of the currently visible region and issues parallel requests to the backend to retrieve the necessary geospatial data. These requests are forwarded by the API Gateway to the `MapDiscoveryManager`.

The `MapDiscoveryManager` queries the `external map service` to obtain the latest map dat of the road network after doing so it uses the `RoutingDBMS` (the graph-oriented database) to fetch the `StreetSegment`s located within the specified bounding box to retrieve the overlay information, which includes aggregated contributions such as obstacles, roadworks, and path quality scores, ensuring the map reflects the real-time condition of the road network.

The diagram also illustrates specific user interactions that can occur within the map view.

If the User taps on a specific path segment, the Client App invokes the `/map/getSegmentDetails` interface. The `MapDiscoveryManager` retrieves the detailed attributes and the list of active contributions for that specific segment from the database and returns them to the client for inspection.

If the User initiates a search for a specific route, the system invokes the `Path Planner` component (refer to the *Search Path* sequence diagram for details). Finally, if the User decides to follow a suggested path, the system redirects the User to the "Automatic Trip Mode" interface, triggering the logic described in the *Follow Path* and *Automatic Insertion* use cases.

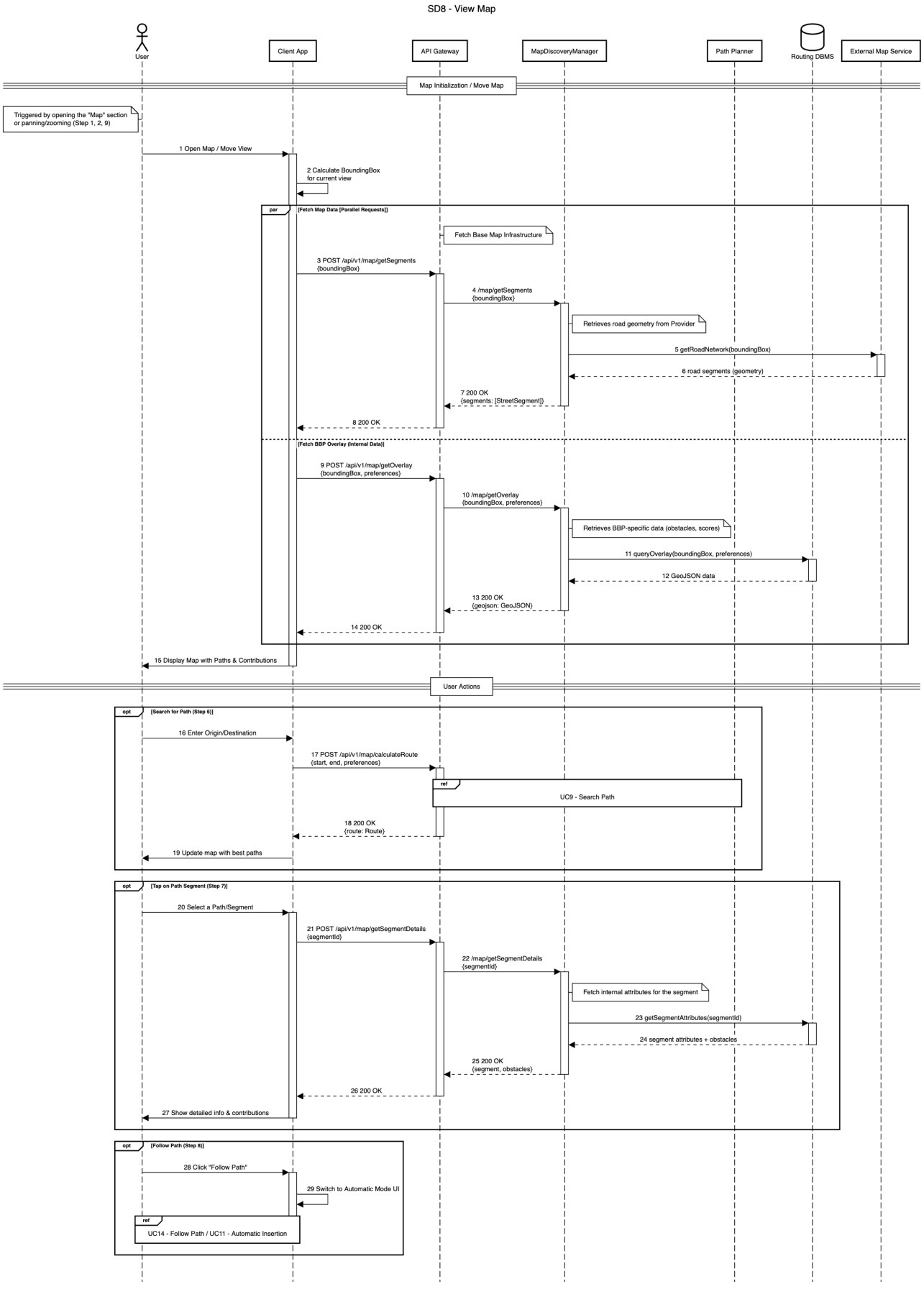


Figure 2.11: View Map

SD9. Search Path

The interaction begins when the User specifies an origin and a destination within the Client App. The request is sent to the `PathFinder` service via the API Gateway. The `Path Planner` first queries the `RoutingDBMS` to retrieve all geometrically valid paths connecting the two points.

If no bike path exists connecting the specified points, the system returns an empty result, and the Client App displays an informative message to the User to notify them that no route is available.

Conversely, if valid paths are found, the system determines the ranking logic based on the user's registration status. If the user is Unregistered, the paths are ranked using standard criteria (e.g., shortest distance or highest safety score) and returned immediately.

If the user is Registered, the `PathFinder` orchestrates a personalization process. It invokes the `ProfileManager` to retrieve the user's specific preferences (e.g., surface type, safety priority) and the `TripHistoryManager` to fetch data on recently completed trips. Using this contextual data, the `PathPlanner` reorders the candidate paths to prioritize those that best match the user's habits and settings. Finally, the ordered list of paths is returned to the Client App for display.

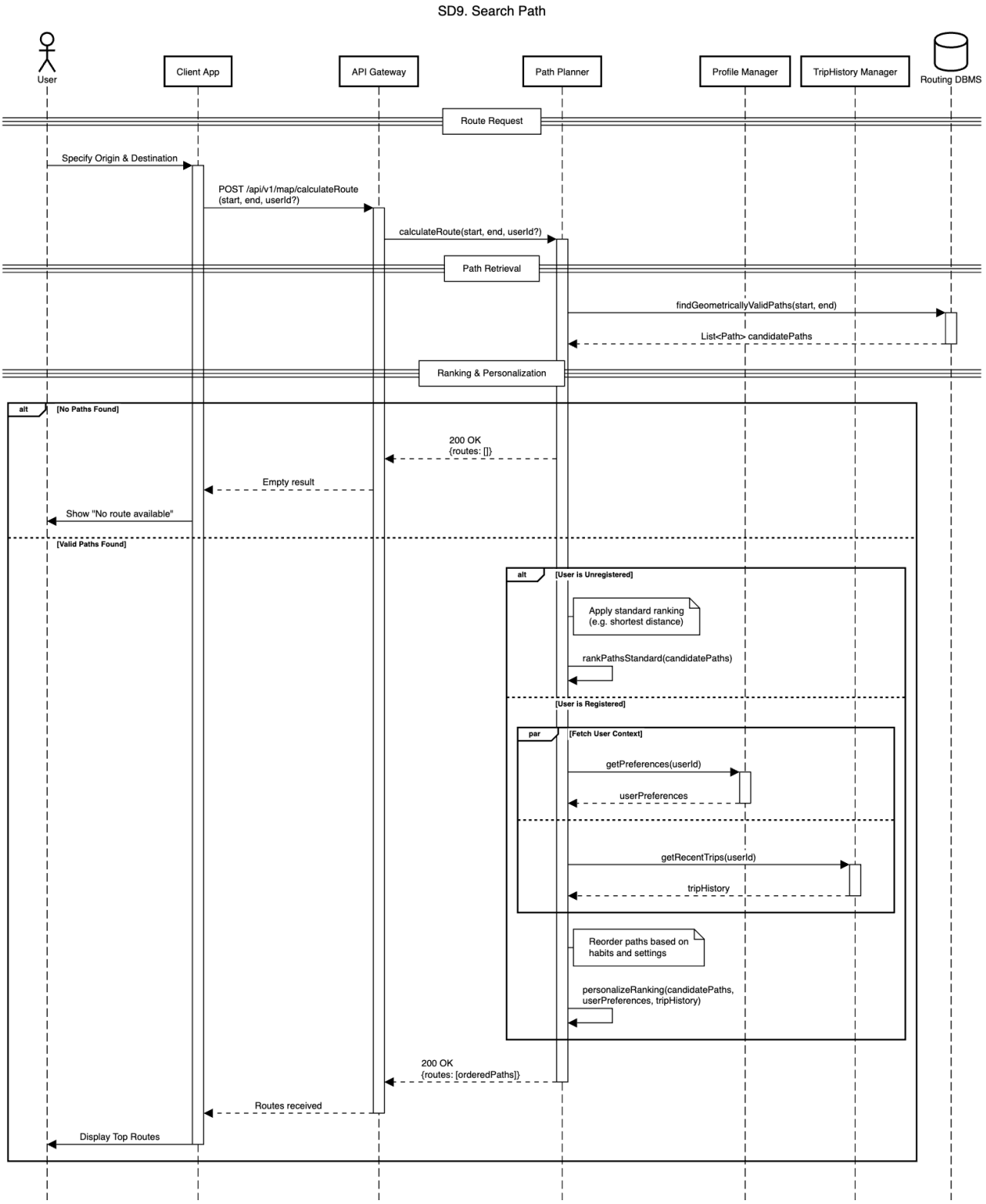


Figure 2.12: Search Path

SD10. Manual Insertion

The interaction begins when the Registered User enters the manual insertion mode, triggering the Client App to initialize a local `TripDraft` state. The user constructs the route iteratively by adding street segments; this allows for two distinct interaction patterns: searching for existing paths (which triggers the logic described in the *Search Path* sequence diagram) or manually selecting segments on the map. In the latter case, the Client App queries the `MapDiscoveryManager` via the API Gateway to retrieve the necessary segment details.

Once the route definition is confirmed, the Client App locally validates the geometric consistency of the draft. If valid, the user can enrich the trip by adding contributions (e.g., obstacles or roadworks). Each contribution undergoes validation to ensure all mandatory fields are present before being attached to the draft.

The process concludes with the user selecting a final action. If the user chooses to *Save as Trip*, the data is sent to the `ManualReportManager` with the `isTrip` flag set to true. The manager persists the trip in the `RawDataDBMS`. If the user marks the trip as public, the service emits a `TripPublishedEvent` to the Kafka Broker to trigger asynchronous aggregation.

Alternatively, if the user chooses to *Publish as Report*, the system treats the input as a standalone contribution. The `ManualReportManager` stores it and immediately publishes a `ContributionPublishedEvent` to Kafka, allowing the `Data Aggregation Service` to process it independently of a trip session. Finally, the user may choose to *Discard* the draft, clearing all local data without backend interaction.

Exceptions, such as invalid draft geometries are handled by displaying appropriate error messages to the user, allowing them to correct the data or retry the operation.

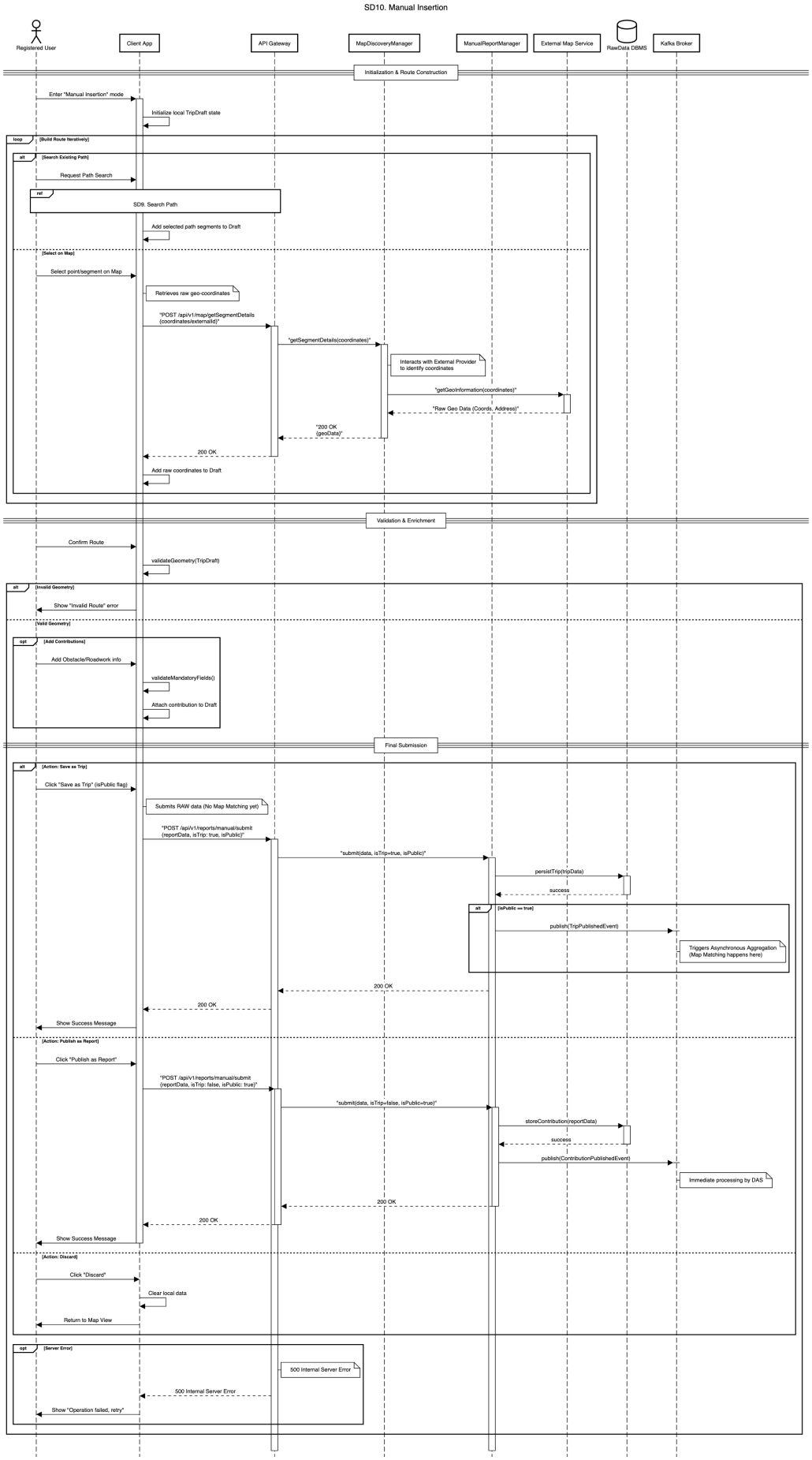


Figure 2.13: Manual Insertion

SD11. Automatic Insertion

The interaction is initiated by the Registered User pressing the "Start recording" button. The Client App performs a preliminary check on hardware availability; if successful, it initializes a local `RecordingSession` and activates the GPS, accelerometer, and gyroscope sensors.

During the recording phase, the Client App periodically acquires sensor data and buffers it locally. While the Use Case description mentions continuous inference, the architectural design delegates the heavy computational logic to the backend to optimize mobile resources. However, the Client App performs real-time monitoring of the user's movement to detect interruptions (stationary status) based on GPS position changes over a configurable time window.

When the user stops the recording, the Client App halts the sensors and submits the collected raw data to the `AutomaticTripProcessor` via the API Gateway. The backend processor reconstructs the full path and calculates trip statistics (e.g., distance, average speed).

Subsequently, the `AutomaticTripProcessor` invokes the `Inference Engine` to analyze the raw sensor logs and detect potential obstacles (e.g., potholes, bumps), generating a set of draft Contributions. It also orchestrates the data enrichment process by triggering the logic described in the *Enrich Trip with Weather Information* sequence diagram.

Finally, the processed trip and the inferred contributions are stored as a draft in the `RawDataDBMS`, and the result is returned to the Client App. The system then redirects the user to the "Review and Edit" screen (refer to *UC13 - Edit Trip*), allowing them to validate or modify the automatically generated data.

Exceptions related to sensor unavailability are handled locally by the Client App, preventing the recording from starting.

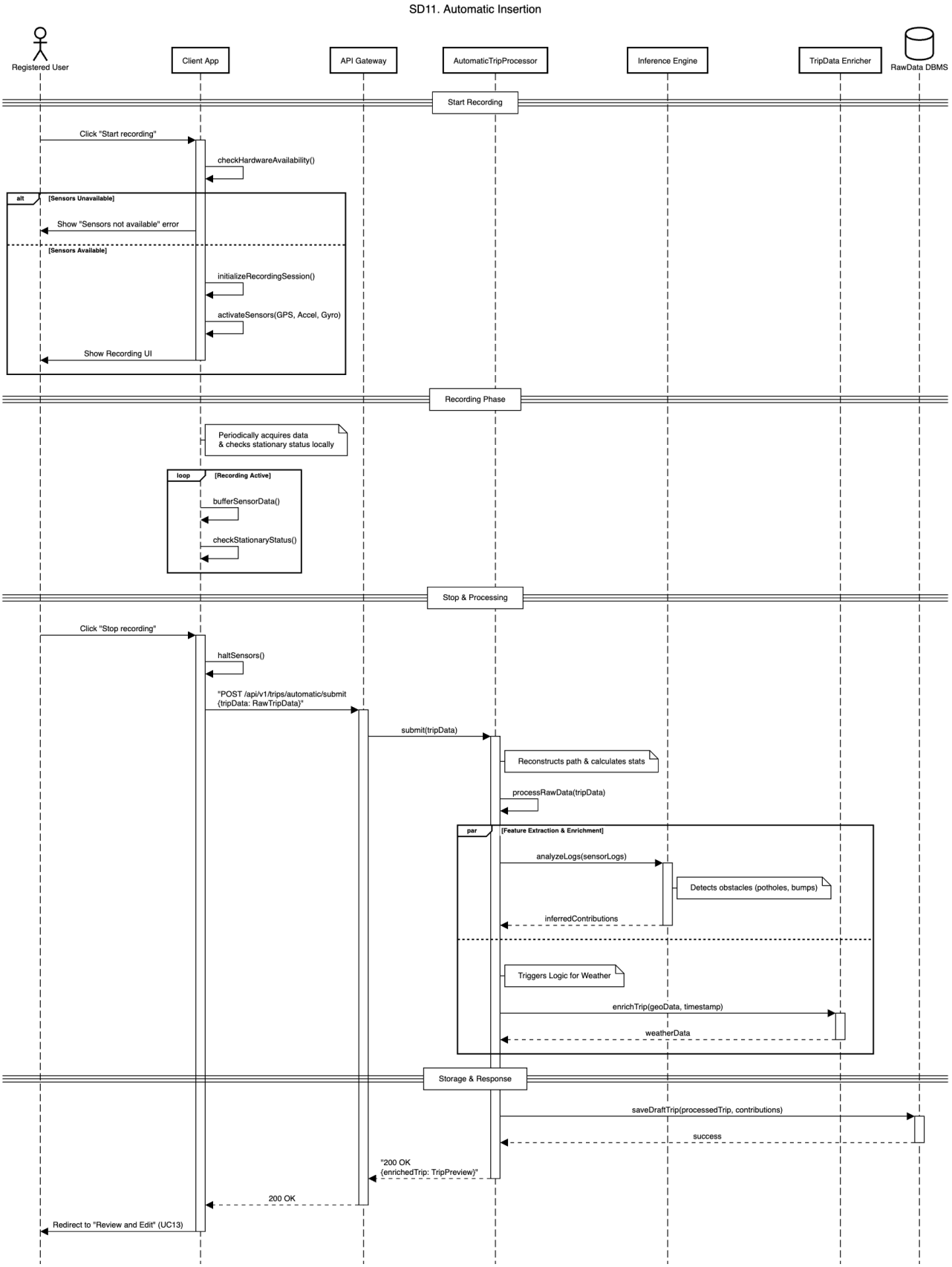


Figure 2.14: Automatic Insertion

SD12. Enrich Trip with Weather Information

This process is typically invoked by the `AutomaticTripProcessor` immediately after reconstructing the path of a completed trip. The `Data Enrichment Service` receives the trip draft containing the GPS trace and the corresponding timestamps.

First, the service validates the input data to ensure the GPS trace contains sufficient information to locate the trip geographically and temporally. If the data is valid, the service constructs a request and queries the external `Weather Service` API to retrieve historical weather conditions (e.g., temperature, wind speed, precipitation) matching the trip's time and location.

If the external service responds successfully, the retrieved meteorological data is parsed and attached to the trip object, enriching the final summary presented to the user.

However, if the external service is unavailable, returns an error, or if the initial GPS data is incomplete, the system adopts a graceful degradation strategy. The trip is marked as "No Weather Data Available," and the process completes without the enrichment, ensuring that the core trip data is preserved and can still be saved successfully.

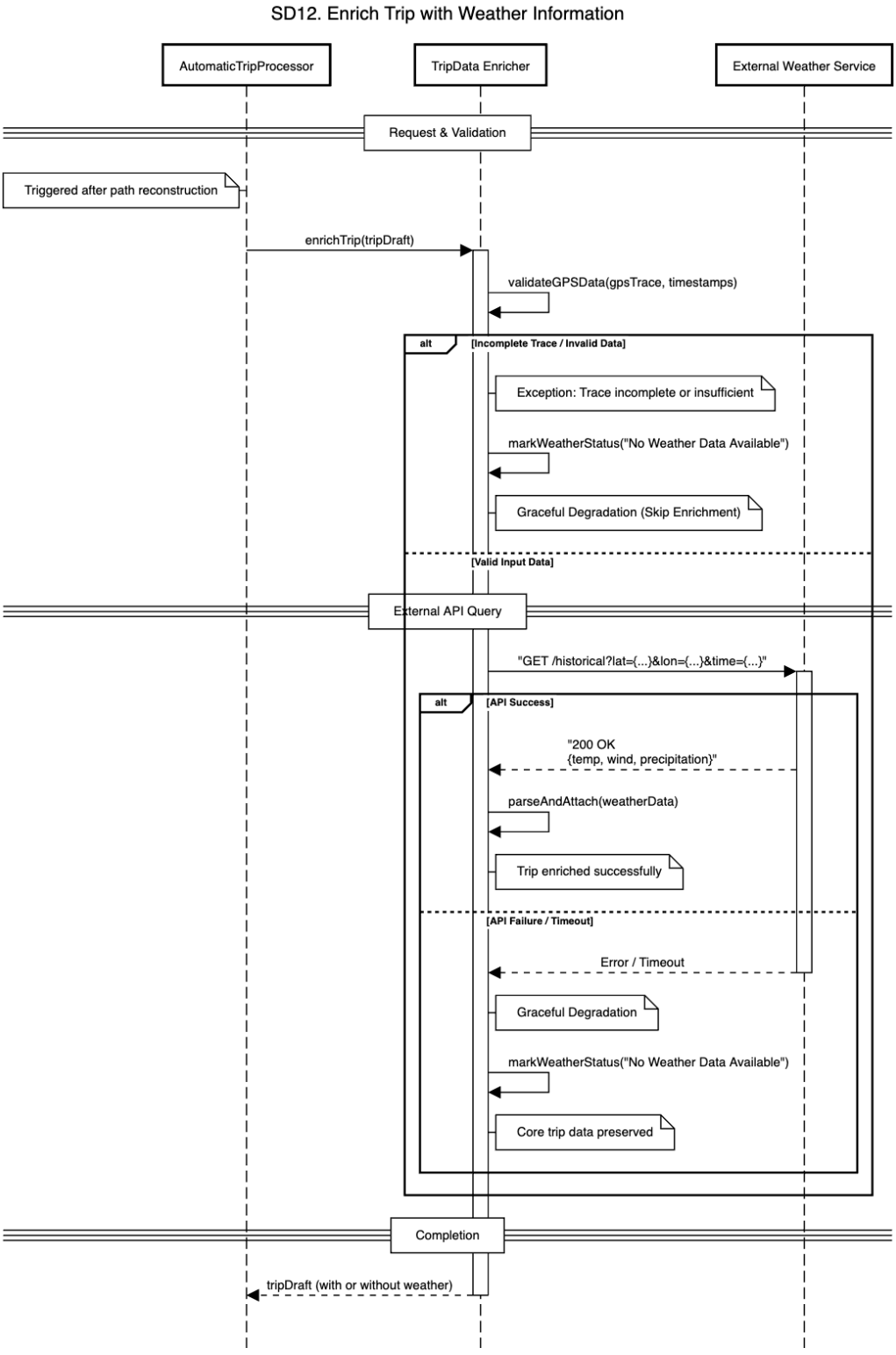


Figure 2.15: Enrich Trip with Weather Information

SD13. Edit Trip

This interaction begins immediately after an automatic trip recording concludes. The Client App presents the user with a review screen displaying the reconstructed path and the set of automatically detected obstacles (e.g., potholes, bumps).

The user can interactively modify the draft data locally: detected obstacles can be confirmed, adjusted, or removed if incorrect. Additionally, the user can manually add missing contributions that the automatic inference might have missed. These changes are maintained in the local application state until a final decision is made.

Once the review is complete, the user selects a final action. If **Store and Publish** is chosen, the finalized data is sent to the `AutomaticTripProcessor`. The processor updates the trip record in the `RawDataDBMS` and marks it as public. Crucially, it then iterates through the valid contributions and the trip itself, emitting corresponding events to the Kafka Broker to trigger the asynchronous aggregation process.

If the user selects **Store Only**, the updated trip data is persisted in the database but marked as private, skipping the publication to Kafka. If **Discard** is chosen, the backend is instructed to permanently delete the draft trip and all associated data from the database.

Exceptions, such as database write failures, are handled by returning an error response to the client, prompting the user to retry the operation to prevent data loss.

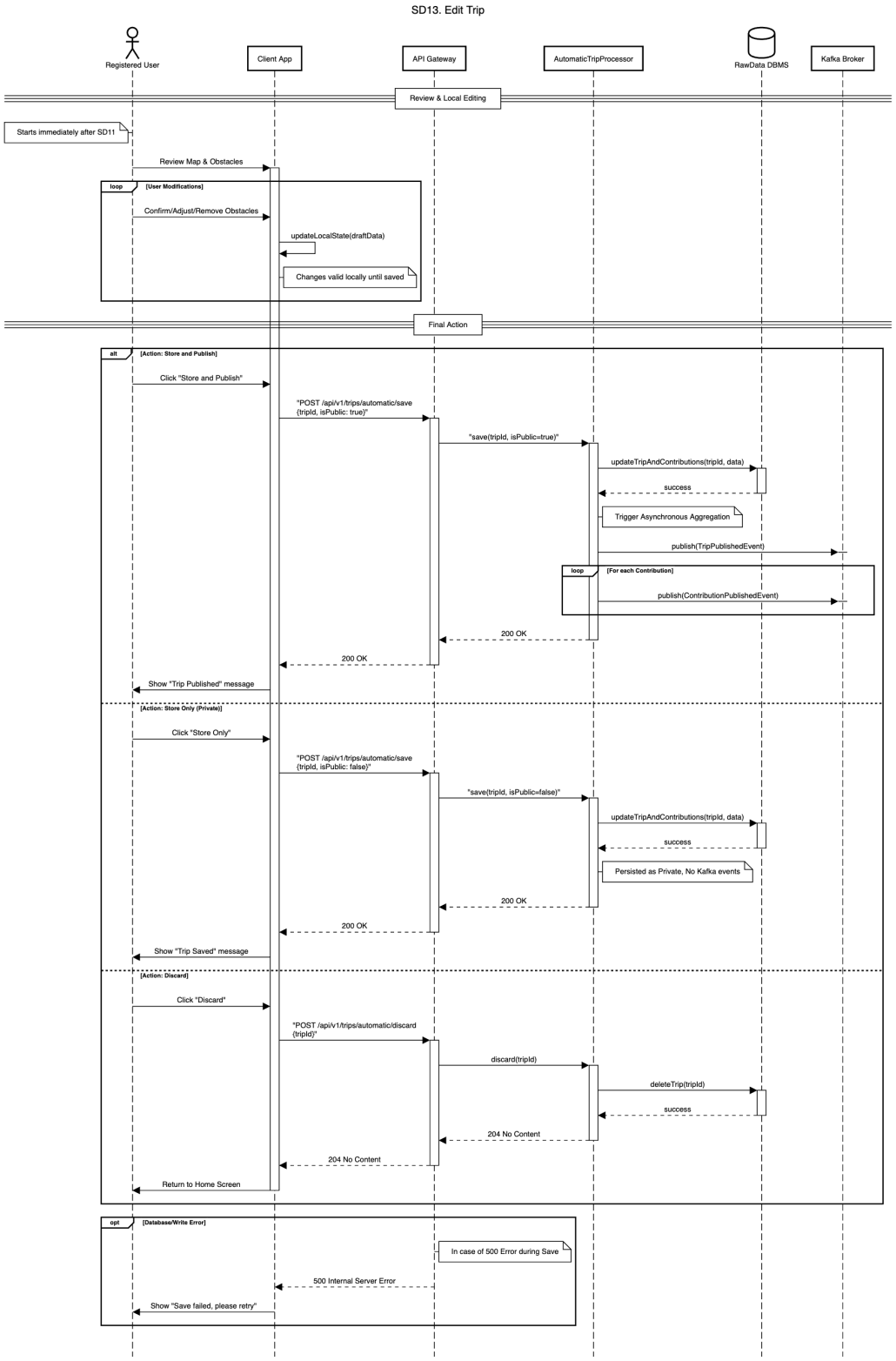


Figure 2.16: Edit Trip

2.6. Selected Architectural Styles and Patterns

For BBP, various styles and patterns allow for better management of the key aspects of the system:

- **Polyglot Persistence**: BBP uses different data storage technologies depending on the specific needs of each subsystem, utilizing relational databases to handle profile and security data, graph DBs to store route data and document databases for the rest. This choice enables the system to optimize performance, scalability, and data modeling by selecting the most appropriate database technology for each domain. As a result, schema evolution and performance tuning can be handled independently, reducing the impact of changes and improving overall system flexibility.
- **Eventual Consistency**: BBP is designed as a distributed system, where strong consistency across all components would negatively impact availability and fault tolerance. According to the CAP theorem, it is impossible to guarantee consistency, availability, and partition tolerance simultaneously. BBP prioritizes availability and partition tolerance, adopting an eventual consistency model temporary inconsistencies in data such as not immediately updated routes are acceptable and do not compromise the correctness of the system's behavior.
- **Local Caching for Map Matching**: To reduce latency and limit the dependency on external map services, the component responsible for map matching (*Map Matching Engine*) adopts a local caching strategy.

  The cache stores associations between spatial coordinate ranges and the corresponding street segments previously resolved. When new GPS data falls within a cached range, the system can reuse the stored mapping instead of issuing a new external API request. This approach significantly improves performance for frequent and geographically localized requests, while also increasing system robustness in the presence of temporary unavailability or rate limiting of external map providers.
- **API Gateway Pattern**: The API Gateway pattern is adopted as a single entry point for all client requests. Its implementation can rely on widely adopted reverse proxy and gateway solutions, without constraining the architecture to a specific technology. For instance, such a gateway could be implemented using a reverse proxy solution (e.g., NGINX), although alternative technologies are equally suitable.
- **Circuit Breaker Pattern**: The Circuit Breaker pattern is applied at the API Gateway level to protect the system from cascading failures caused by unavailable or slow

downstream services, including external providers. Although BBP does not offer safety-critical functionalities, this mechanism improves system robustness and availability by isolating failures and enabling graceful degradation, for instance by temporarily disabling non-essential features such as weather enrichment.

- **Asynchronous Processing via Message Broker**: To decouple time-sensitive operations from computationally expensive background tasks, BBP adopts an asynchronous messaging approach. In particular, the persistence of recorded trips is handled synchronously to ensure a responsive user experience, while the subsequent merging and aggregation of path information is triggered asynchronously through events published to a message broker. This design follows an event-driven architecture, improving scalability and fault isolation by allowing the merging process to evolve independently from trip ingestion. Temporary delays in the merging phase do not compromise system correctness, as path aggregation naturally tolerates eventual consistency.

3 User Interface Design

3.1. General overview

The design of the User Interface for the BBP app has a vital role in ensuring a clear and friendly user experience. The following section contains a comprehensive look at the app and its core functionalities through the use of a mock-up UI highlighting the features that make the app as intuitive and easy to use.

The UI features a clean and consistent layout, allowing users to quickly access core features easily and understand how to navigate each section without difficulty.

UI mock-ups are not provided for every page described in the DD document; only the most relevant sections and their functionalities are showcased. For simplicity, this document does not include pop ups, although some may be used in the app itself.

Design choices, fonts and colors may be modified to follow business decisions to improve user engagement.

Each app screen is responsive and adapts to all user devices

3.2. User Interface Layouts

3.2.1. User Authentication

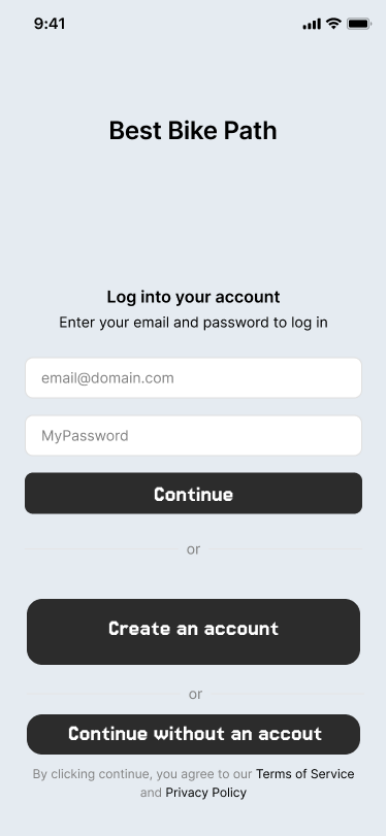


Figure 3.1: Sign in

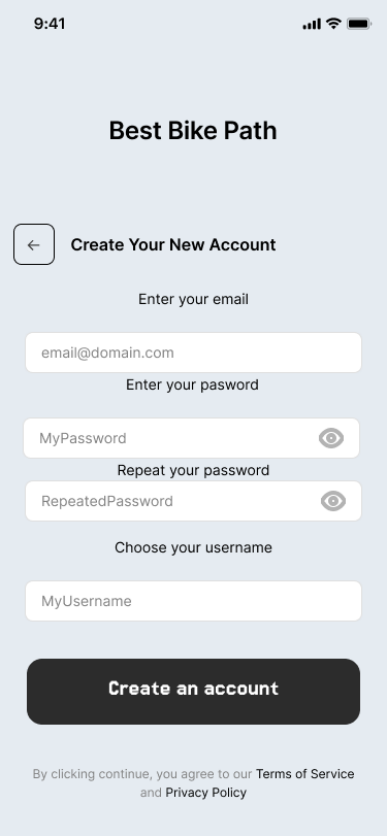


Figure 3.2: Sign up

Figure 3.1 shows the screen displayed when a user opens the BBP app for the first time. From this screen, users can either log into an existing account by entering their credentials and pressing
textitContinue, or create a new account by selecting the
textitCreate an account button. Alternatively, users may choose to access the application with limited functionalities by pressing
textitContinue without an account.

Figure 3.2 illustrates the account creation screen, where a user can create a new BBP account by providing all the required information and pressing
textitCreate an account. If the user wishes to return to the sign-in screen, they can do so by pressing the arrow located in the top-left corner of the screen.

3.2.2. Map and navigation

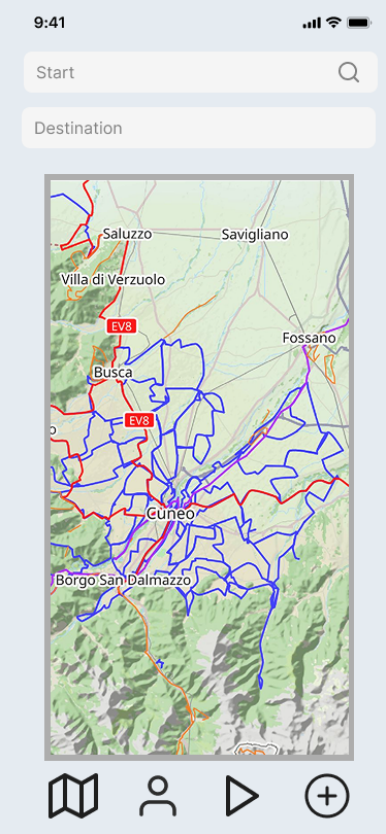


Figure 3.3: Map

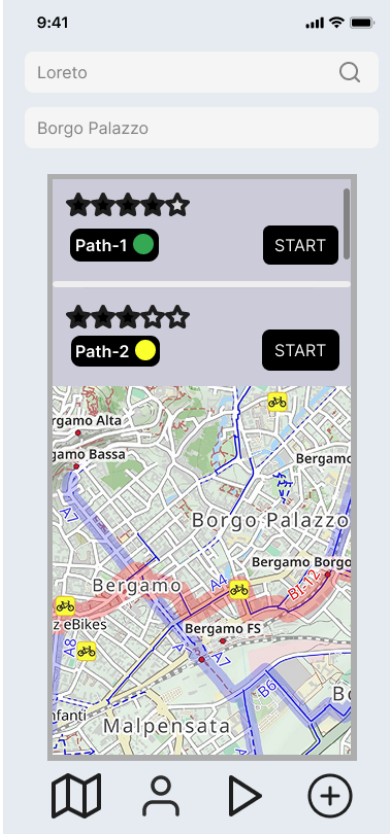


Figure 3.4: Navigation

This section describes what a user sees after authentication, either when opening the app from the second time onward or by selecting the map icon in the bottom navigation bar. Figure 3.3 represents the standard map view, showing all the available bike paths in the selected area. The user can zoom in and out and move the map using touch gestures.

Figure 3.4 illustrates the navigation view displayed after the user specifies an origin and a destination using the search bar at the top of the screen. Below the map, the system presents the best available paths ranked according to their score, together with the corresponding score and average road quality. The top-ranked path is highlighted in red on the map. The user can start following a selected path by pressing
textitStart on the right side of the corresponding path entry.

3.2.3. Profile

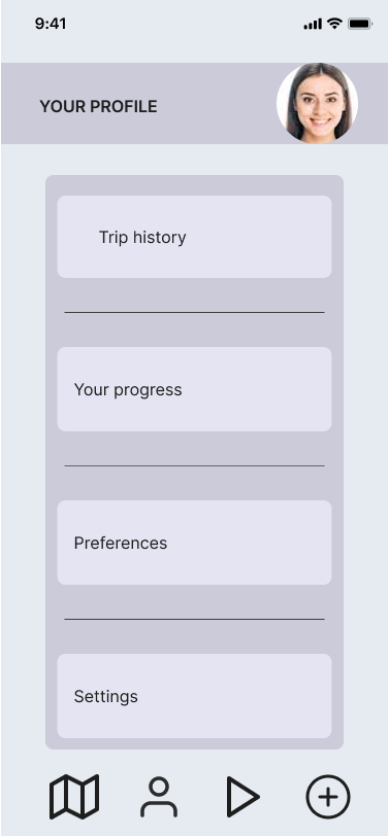


Figure 3.5: Profile

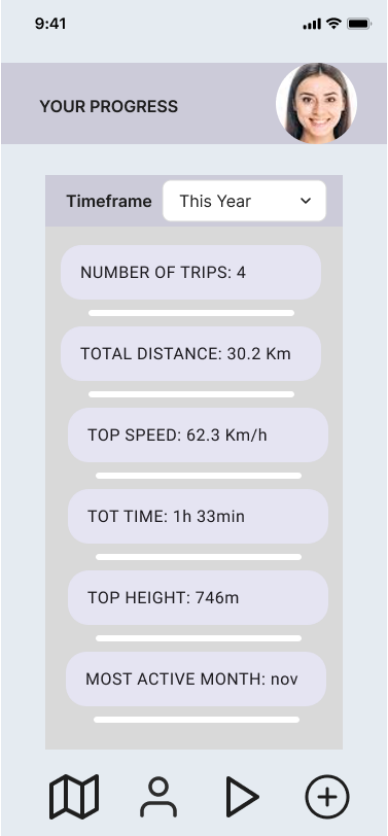


Figure 3.6: Activity

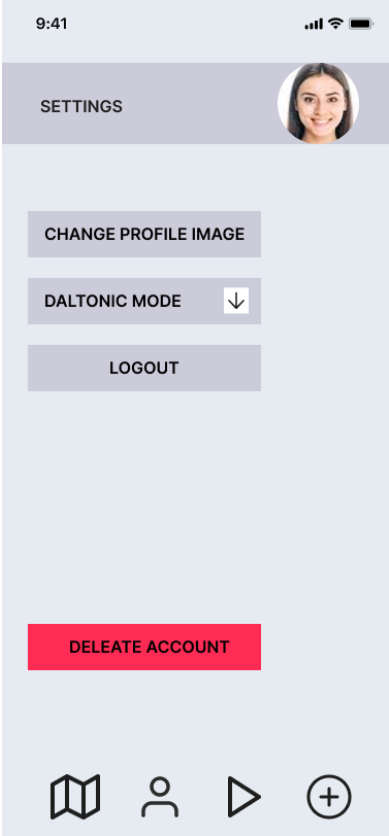


Figure 3.7: Settings

The profile interface can be accessed by selecting the human-shaped icon in the bottom navigation bar. From this screen, pressing the
textitTrip history button allows the user to access all information related to their recorded trips, including detailed statistics and the option to publish them.

As shown in Figure 3.6, by selecting
textitYour progress the user can view an overview of their activity, including metrics such as the total number of completed trips and the cumulative distance traveled. These statistics can be further filtered by a selected time frame, such as
textitThis year.

The
textitPreferences section allows the user to specify preferred road types and disliked obstacles, enabling BBP to personalize the path scoring system and provide a more tailored user experience for registered users. As illustrated in Figure 3.7, by selecting
textitSettings the user can modify basic account settings, such as the profile image, or perform actions including logging out or deleting the account.

3.2.4. Trip history

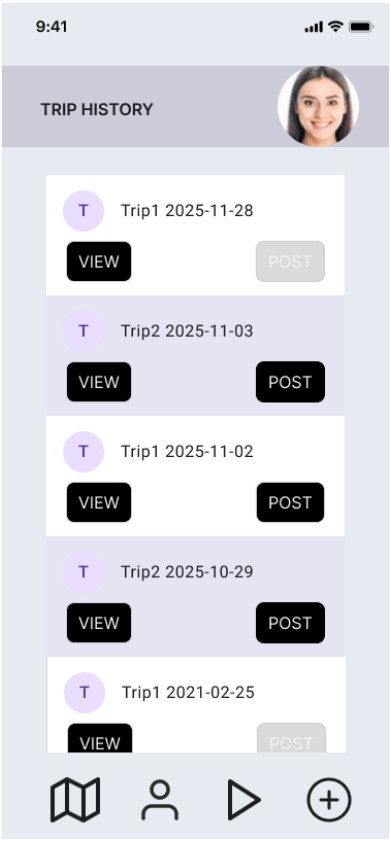


Figure 3.8: History

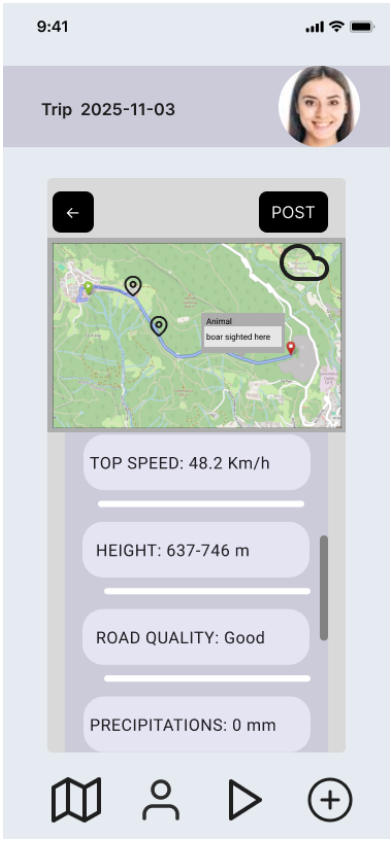


Figure 3.9: Trip data

Figure 3.8 shows the interface displayed when a user selects
textitTrip history from the profile section. From this screen, the user can view the list of all completed trips in chronological order. By pressing
textitPost, the user can publish the contributions associated with a trip; this button is disabled if the trip has already been published.

Figure 3.9 can be accessed by pressing
textitView on any trip in the history list. This screen displays detailed trip statistics together with a map representation of the recorded path. The pins shown on the map represent detected events, and the user can view their details by selecting the corresponding pin.

3.2.5. Automated trips

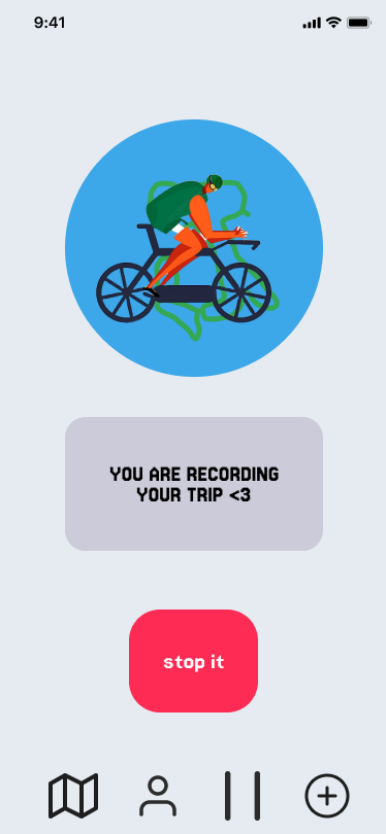


Figure 3.10: Trip recording

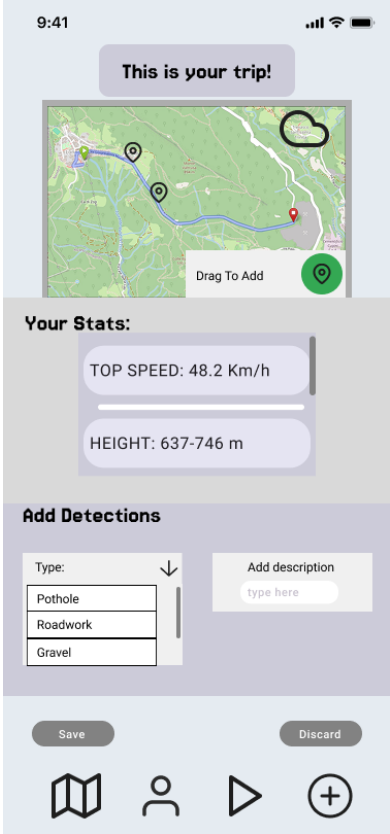


Figure 3.11: modify trip

Figure 3.10 is displayed when a user selects the
textitPlay icon in the bottom navigation bar, starting the automatic trip recording process, during which sensor data and trip statistics are collected. While the recording is active, the user can continue to use the other functionalities of the application. To stop the recording, the user can press the red
textitStop button shown in Figure 3.10 or the same button used to start the recording from any screen.

After the trip is stopped, the user is presented with the interface shown in Figure 3.11, where all recorded statistics can be visualized and additional detections can be added by selecting a category and dragging a green pin to the desired location. Detections can also be removed by selecting them and pressing the
textitX button that appears. The user may then save the trip to their history, with the option to publish it immediately or at a later time, or discard the trip entirely by selecting the corresponding action.

3.2.6. Manual trip Insertion



Figure 3.12:
Path generation

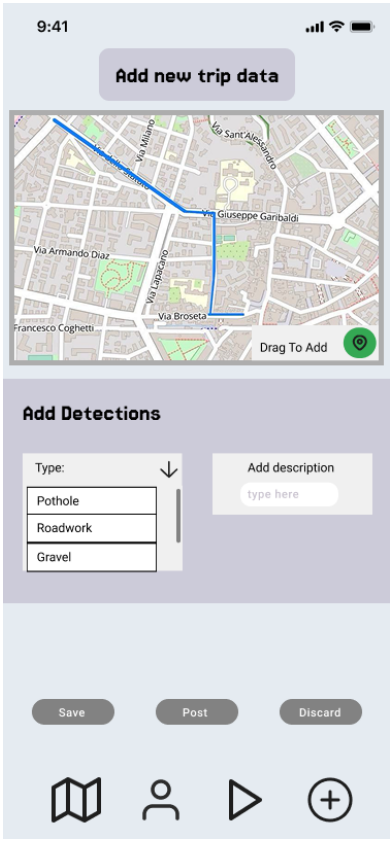


Figure 3.13:
Detection generation

Figure 3.12 is displayed when a user selects the
textit+ button in the bottom navigation bar, initiating the manual trip creation process. From this screen, the user can add street segments by selecting them directly on the map, entering the name of a street, or searching for an entire path by specifying a starting point and a destination. Once the desired path is defined, it is displayed in blue, and the user can proceed by pressing the
textitConfirm path button.

After confirming the path, the interface shown in Figure 3.13 is displayed, allowing the user to add detections using the same interaction methods described in the previous section. Once all desired detections have been added, the user may save the trip to their history, publish it solely as a contribution without storing the trip in the history, or discard the trip entirely by selecting the corresponding option.

4 Requirements Traceability

4.1. Functional Requirements Traceability

The following table maps the functional requirements defined in the RASD document to the components identified in the Component Diagram. Requirements are reported verbatim as defined in the RASD. When a requirement describes a complex or cross-cutting behavior, it is intentionally mapped to multiple components that cooperatively contribute to its fulfillment. External systems are included in the mapping when they provide essential support to the realization of a requirement. For requirements involving the publication and consolidation of contributions, the addition of information to the public dataset is performed asynchronously through event-driven processing.

Component	Mapped Requirements
SecurityManager	[R1] Upon request, the system shall allow a new User to sign up to the platform, as long as all required personal information is provided and no existing account is associated with the submitted email. [R2] Upon request, the system shall allow the User to log into the platform, as long as authentication succeeds, and access to his personal profile.
ProfileManager	[R3] Upon request, the system shall allow a Registered User to update their profile information, as long as all necessary data is provided. [R4] Whenever a Registered User updates their preferences, the system shall store these changes and apply them to future personalized route recommendations.
AutomaticTripProcessor	[R5] Upon request, the system shall allow a Registered User to manually start recording a trip, as long as GPS permissions are granted. [R6] Upon request, the system shall allow a Registered User to manually stop a trip that was previously started. [R7] Whenever an automatic trip is recorded, the system shall store the trip with its GPS track, timestamps, speed data and contributions into the User's trip history. [R10] Upon completion of an automatic trip, the system shall present the detected path and all detected obstacles to the User, requesting confirmation or correction.

	[R11] Whenever a Registered User confirms or corrects automatically detected information, the system shall convert such detections into contributions associated with the current path. [R16] Upon request, the system shall allow a Registered User to publish any of their contributions (manual or confirmed automatic) that are currently in draft state. [R18] Whenever a recently completed automatic trip ends and its detections are validated, the system shall offer the User an immediate option to publish the contributions generated from that trip.
InferenceEngine	[R9] Whenever automatic detection identifies a potential obstacle, the system shall generate a detection item and store it in the User's trip session.
DataEnrichmentEngine	[R8] Whenever an Automatic Trip is stored, the system shall compute statistics such as total distance, average speed, travel time, and other performance metrics. [R35] Whenever an Automatic trip is saved, the system shall request weather information (temperature, wind speed, weather conditions) from an external service based on trip time and location. [R36] Whenever weather data is retrieved, the system shall associate it with the corresponding trip summary.
ManualReportManager	[R12] Upon request, the system shall allow a Registered User to manually create a Manual trip by specifying the StreetSegments they have cycled through, without using sensor data and without computing trip statistics. [R13] Upon request, the system shall allow a Registered User to manually create a contribution describing information about a specific StreetSegment, regardless of whether it is associated with a manually created trip. [R14] Upon request, the system shall allow a Registered User to publish any of their manual contributions from a stored trip not yet published. [R16] Upon request, the system shall allow a Registered User to publish any of their contributions (manual or confirmed automatic) that are currently in draft state.
TripHistoryManager	[R15] Upon request, the system shall show a Registered User the statistics and contributions of any of their stored trips.

ContributionAnalyzer	[R21] At configurable time intervals, the system shall aggregate all published contributions referring to the same StreetSegment, taking into account data recency, number of supporting contributions, and type of Information, in order to update the StreetSegment's merged information, score, and bike-path status.
MapMatchingEngine	[R19] Whenever a published contribution contains standard Information (e.g., obstacles, roadworks, warnings) about a StreetSegment that is currently considered a bike path, the system shall merge such Information into the StreetSegment's aggregated data used for routing and scoring. [R20] Whenever a published contribution contains BikePathSuitability Information about a StreetSegment, the system shall store it and consider it in the periodic aggregation of bike-path suitability for that StreetSegment. [R21] At configurable time intervals, the system shall aggregate all published contributions referring to the same StreetSegment. [R22] Whenever a published contribution contains only standard Information about a StreetSegment that is not currently marked as a bike path, the system shall discard such Information for aggregation purposes and shall not use it to update that StreetSegment.
RelevanceEngine	[R21] At configurable time intervals, the system shall aggregate all published contributions referring to the same StreetSegment, taking into account data recency, number of supporting contributions, and type of Information, in order to update the StreetSegment's merged information, score, and bike-path status [R23] Whenever the aggregated BikePathSuitability Information for a StreetSegment exceeds a configurable positive threshold, the system shall mark that StreetSegment as a bike path; whenever it falls below a configurable negative threshold, the system shall unmark that StreetSegment as a bike path. [R33] Whenever the aggregated BikePathSuitability for a StreetSegment no longer satisfies the suitability criteria for cycling, the system shall unmark that StreetSegment as a bike path and treat it as a regular street segment.

GraphWeightUpdater	[R19] Whenever a published contribution contains standard Information (e.g., obstacles, roadworks, warnings) about a StreetSegment that is currently considered a bike path, the system shall merge such Information into the StreetSegment's aggregated data used for routing and scoring. [R21] At configurable time intervals, the system shall aggregate all published contributions referring to the same StreetSegment, taking into account data recency, number of supporting contributions, and type of Information, in order to update the StreetSegment's merged information, score, and bike-path status [R34] Whenever the bike-path status or aggregated condition of one or more StreetSegments changes, subsequent route computations shall use the updated information.
Path Planner	[R25] Upon request, the system shall allow the User to specify an origin and a destination for route lookup. [R26] Whenever the system receives an origin and a destination, it shall compute candidate Paths by running a routing algorithm over the StreetSegments that are currently marked as bike paths. [R27] Whenever the system receives an origin and a destination, it shall display the top 5 candidate Paths ranked by score. [R28] If no fully bike-path-only route exists between the selected origin and destination, the system shall still display the map with all available bike-path StreetSegments highlighted, allowing the User to inspect partial bike-path options. [R31] Whenever a Registered User requests a route, the system shall tailor the ranking of available paths according to the User's saved preferences. [R32] Whenever path scoring is computed, the system shall consider path condition, number and type of obstacles, consensus level from contributions, efficiency toward destination, and traffic comfort indicators. [R34] Whenever the bike-path status or aggregated condition of one or more StreetSegments changes, subsequent route computations shall use the updated information.

MapDiscoveryManager	[R24] Upon request, the system shall allow any User (registered or unregistered) to browse the map and visually inspect all StreetSegments currently marked as bike paths. [R29] Upon request, the system shall show detailed information about a selected Path, including the sequence of StreetSegments, their status, obstacles, and the aggregated user contributions associated with them. [R30] The system shall visually highlight on the map only those StreetSegments that are currently marked as bike paths, distinguishing them from regular street segments.
Kafka Message Broker	[R17] Whenever a Registered User publishes a contribution, the system shall add all its Information items to the public dataset associated with the corresponding StreetSegment.
External Map Service	[R24] Upon request, the system shall allow any User (registered or unregistered) to browse the map and visually inspect all StreetSegments currently marked as bike paths. [R25] Upon request, the system shall allow the User to specify an origin and a destination for route lookup. [R26] Whenever the system receives an origin and a destination, it shall compute candidate Paths by running a routing algorithm over the StreetSegments that are currently marked as bike paths. [R29] Upon request, the system shall show detailed information about a selected Path, including the sequence of StreetSegments, their status, obstacles, and the aggregated user contributions associated with them.
Weather Service (External System)	[R35] Whenever an Automatic trip is saved, the system shall request weather information (temperature, wind speed, weather conditions) from an external service based on trip time and location. [R36] Whenever weather data is retrieved, the system shall associate it with the corresponding trip summary.
Mailing Service (External System)	[R1] Upon request, the system shall allow a new User to sign up to the platform, as long as all required personal information is provided and no existing account is associated with the submitted email.

Table 4.1: Mapping between Functional Requirements and System Components.

4.2. Non-Functional Requirements Traceability

4.2.1. Performance Requirements

The performance requirements defined in Section 3.3 *Performance Requirements* of the RASD are satisfied through specific architectural and design decisions adopted in the BBP system.

First, the mobile client is intentionally designed as a lightweight component. Its responsibilities are limited to user interaction management, map rendering, and acquisition of GPS and sensor data from the device. Computationally intensive operations, such as map matching, contribution merging, and route computation, are delegated to backend services. This separation allows the system to provide a responsive user interface and to support real-time trip recording without application freezes or noticeable delays, as required by the RASD.

The merge process of user contributions, which is particularly computationally expensive due to repeated map matching and aggregation operations, is handled asynchronously through a message broker. This design choice decouples heavy processing from synchronous user requests and allows merge operations to be executed outside peak load periods. As a result, system responsiveness is preserved even under high traffic conditions.

Scalability requirements, including the ability to handle up to 15,000 concurrent users, are addressed through the presence of an orchestrator managing the backend services. The orchestrator enables horizontal scaling and elastic resource allocation, dynamically adapting the number of service instances based on the current workload. This ensures that core functionalities, such as map visualization, path computation, and trip recording, remain available without significant performance degradation.

Route computation efficiency is further improved by adopting a custom graph-based data model that includes only relevant street segments. This representation allows the use of efficient pathfinding algorithms, such as A*, with a domain-specific heuristic based on edge weights that reflect the status and quality of street segments. By operating on a reduced and specialized graph, the system significantly lowers computational complexity and improves response times for path search operations.

To further optimize performance, caching mechanisms are introduced, particularly for map matching results and frequently accessed street segments. By reusing previously computed mappings for commonly traversed areas, the system reduces the number of external API calls and avoids redundant computations, leading to faster processing of new contributions.

During automatic trip recording, raw sensor and GPS data are immediately persisted and transmitted to the backend without requiring synchronous execution of map matching or merge

operations. These tasks are performed asynchronously, ensuring uninterrupted data acquisition on the client side and preventing data loss under varying network conditions.

Finally, asynchronous data synchronization guarantees that uploaded trips and contributions become available to users within a bounded and controlled time after upload completion. In particular, the adopted design ensures that newly recorded data is processed and made accessible within 30 seconds, in compliance with the synchronization timeliness requirement specified in the RASD.

4.2.2. Software System Attributes

The software system attributes introduced in Section 3.5 *Software System Attributes* of the RASD are achieved through a combination of architectural decisions and runtime mechanisms specifically designed for a mobile-centered and distributed system such as BBP. The following paragraphs describe how these attributes are supported by the proposed design.

- **Reliability:** The reliability of the BBP system is ensured by decoupling critical data acquisition from backend processing. Trip data and sensor readings are collected and temporarily stored on the mobile device, allowing recording activities to continue even in the presence of temporary network disruptions or application restarts. Backend services process uploaded data asynchronously, reducing the impact of transient failures. Furthermore, service replication and automated recovery mechanisms managed by the orchestration platform increase fault tolerance, enabling the system to continue operating and to degrade gracefully when partial failures occur.
- **Availability:** System availability is supported through the replication and horizontal scaling of backend services, which are dynamically managed to handle variations in workload. In addition, the mobile application follows an offline-first approach: core functionalities such as trip recording and local data visualization remain usable even when backend services are temporarily unreachable. Once connectivity is restored, deferred synchronization ensures consistency without requiring user intervention.
- **Security:** Security is achieved by enforcing authentication and authorization mechanisms based on secure access tokens for all interactions with backend APIs. Communication between the mobile client and the backend is protected using encrypted channels based on HTTPS and TLS. Access to sensitive device resources, such as location and motion sensors, is explicitly controlled through user-granted permissions in accordance with the underlying mobile operating system security model.
- **Maintainability:** Maintainability is promoted by structuring the backend as a set of

independent services, each responsible for a well-defined subset of system functionalities. This modular organization allows individual services to be modified, extended, or replaced with minimal impact on the rest of the system. z

- **Portability:** Portability is ensured by designing the mobile client to support multiple operating systems, namely Android and iOS, while isolating platform-specific components from the core application logic. On the backend side, services are deployed using containerized technologies and communicate through standardized, technology-agnostic interfaces. This combination enables the system to be deployed and migrated across different execution environments with limited adaptation effort.

5 | Implementation, Integration and Test Plan

5.1. Overview

This section outlines the strategy for the development, integration, and testing of the *Best Bike Paths* (BBP) system. Given the architectural choice of a microservices-inspired system supported by a four-tier structure (as detailed in Chapter 2), the implementation and integration phases are not distinct, sequential blocks. Instead, they are intertwined processes designed to maximize efficiency and allow parallel development.

The modular nature of the system—comprising distinct components like the *Automatic Trip Processor*, *Data Aggregation Service*, and *Map Service*—allows different development teams to work simultaneously on different "bounded contexts." However, this independence necessitates a rigorous integration testing strategy to ensure that the asynchronous communication via the Kafka Message Broker and the synchronous REST API interactions function correctly.

The testing process will follow a bottom-up approach: starting from Unit Testing of individual components, moving to Integration Testing between interacting services, and culminating in System (End-to-End) Testing to validate the requirements defined in the RASD.

5.2. Development Plan

The development plan is structured to prioritize the critical path of the application: the ability to ingest data, process it to update the road network state, and serve this data to the user. The components are grouped into logical subsystems to be implemented in the following order:

5.2.1. Phase 1: Infrastructure and Core Services

The foundation of the system must be established first to handle user identity and request routing.

- **Infrastructure Setup:** Deployment of the *API Gateway* (NGINX), the *Kafka Message Broker*, and the configuration of the DBMS instances (*SecurityDBMS*, *ProfileDBMS*, *RawDataDBMS*, *RoutingDBMS*).
- **User Management:** Implementation of the **Security Manager** and **Profile Manager**. These are prerequisites for any other operation since all interactions (trip recording, reporting) require an authenticated *Registered User*.

Testing Focus: Verification of token generation/validation and correct routing of requests through the Gateway.

5.2.2. Phase 2: Data Acquisition (The "Write" Side)

Without data, the platform offers no value. This phase focuses on the mobile client and the ingestion services.

- **Mobile Application (Sensor Logic):** Implementation of the local recording logic, sensor data buffering (GPS, accelerometer), and local interruption detection.
- **Trip Processing:** Development of the **Automatic Trip Processor** and **Manual Report Manager**. This includes the *Inference Engine* for detecting obstacles from raw sensor data and the *TripData Enricher* for fetching weather data.

Testing Focus: Simulation of sensor data streams to verify obstacle detection algorithms and load testing the ingestion endpoints to ensure they can handle high concurrency.

5.2.3. Phase 3: Data Aggregation (The Core Business Logic)

This is the most critical and complex subsystem, distinguishing BBP from standard mapping tools. It should receive significant development resources.

- **Data Aggregation Service:** Implementation of the **Contribution Analyzer**, **Map Matching Engine**, and **Relevance Engine**.
- **Graph Updates:** Development of the **Graph Weight Updater** to modify the *RoutingDBMS* based on the merged contributions.

Testing Focus: Validation of the merging algorithms (e.g., ensure that multiple reports of a pothole converge into a single confirmed obstacle) and verification of the asynchronous consumption of events from Kafka.

5.2.4. Phase 4: Navigation and Visualization (The "Read" Side)

Once the data is reliably stored and aggregated, the consumption layer is built.

- **Map Service:** Implementation of the **Map Discovery Manager** to overlay BBP data on the external base map.
- **Routing Logic:** Development of the **Path Planner** to compute routes based on the custom weights in the *RoutingDBMS*.

Testing Focus: Verifying that the routes returned actually prioritize "bike-friendly" segments over standard shortest paths, complying with the aggregated scores.

5.3. Integration and Test Strategy

5.3.1. Unit Testing

Each component described in the Component View will undergo strict unit testing.

- **Inference Engine:** Tested with pre-recorded datasets of accelerometer patterns to ensure it correctly classifies potholes vs. normal riding.
- **Path Planner:** Tested with graph mock-ups to verify the correctness of the routing algorithm (e.g., Dijkstra or A*) using custom weights.
- **External Connectors:** Mocks will be used to simulate the *Weather Service* and *External Map Service* responses to handle edge cases (e.g., service unavailability) without making actual network calls.

5.3.2. Integration Testing

Integration testing addresses the interaction between components and the persistence layer.

- **API Contract Testing:** Verifying that the REST APIs exposed by the microservices (e.g., `/trips/automatic/submit`) adhere to the JSON schemas defined in the Design Document.
- **Asynchronous Messaging Testing:** Verifying the *Producer-Consumer* flow via Kafka. For example, ensuring that a `TripPublishedEvent` emitted by the *Automatic Trip Processor* is correctly consumed by the *Contribution Analyzer*.
- **Database Integration:** Ensuring that the *Map Matching Engine* correctly retrieves segments from the external provider and maps them to the internal graph representation in the *RoutingDBMS*.

5.3.3. System and Acceptance Testing

This phase validates the system against the Functional Requirements (traceability analysis).

- **Scenario-based Testing:** End-to-end execution of the scenarios defined in the RASD. For example, a tester records a trip, reports an obstacle, publishes it, and a second tester verifies that the obstacle appears on the map and affects routing.

- **User Acceptance Testing (UAT):** A beta release to a small group of cyclists to validate the usability of the mobile interface and the relevance of the "Best Path" suggestions in real-world urban environments.

6 Effort Spent

Group Member	Effort Spent in each Section	
Francesco Rausa	Introduction	1h
	Architectural Design	15h
	User Interface Design	1h
	Requirements Traceability	2h
	Implementation, Integration and Test Plan	1h
	Reasoning	7h
Pietro Poggi	Introduction	0h
	Architectural Design	8h
	User Interface Design	13.5h
	Requirements Traceability	0.5h
	Implementation, Integration and Test Plan	0h
	Reasoning	5h
David Ravelli	Introduction	0.5h
	Architectural Design	13h
	User Interface Design	1h
	Requirements Traceability	1h
	Implementation, Integration and Test Plan	3.5h
	Reasoning	7h

Table 6.1: Effort spent by each member of the group.

7 References

7.1. Used Tools

- **Overleaf**: LaTeX collaborative writing.
- **SequenceDiagram.com with IntelliJ IDEA plugin**: sequence diagrams.
- **Lucidchart**: all other diagrams.
- **Figma**: UI Interfaces.
- **ChatGPT**: AI-assisted support.

7.1.1. Use of ChatGPT

- **Textual Refinement**: Used to assist in refining the English grammar, vocabulary, and flow of the document. It is important to note that all architectural decisions, concepts, and logic are original to the authors; the AI tools served solely to rephrase and polish the author-generated content.
- **Graphic Assets Generation**: Used to generate specific service icons for the High-Level Architecture diagram (Section 2.1), as suitable representations were not available in the Lucidchart library.
- **LaTeX Support**: Provided assistance with complex formatting and syntax debugging.
- **Sequence Diagram Generation**: Used to accelerate the initial creation of sequence diagrams by converting detailed textual descriptions of the Use Cases (derived from the Requirements Analysis) into code compatible with the IntelliJ IDEA Sequence Diagram plugin. The generated code served as a preliminary draft; the authors manually reviewed and refined the diagrams to ensure consistency with the system architecture and correct component interactions.

  Some notable prompts used were:

  - **Sequence Diagram Generation (Example: Sign Up)**
    - **Input**: "Given the context of the Design Document provided so far, generate the code for the IntelliJ Sequence Diagram plugin based on the following description:

      The User initiates the sign-up process by clicking the 'Sign Up' button on the homepage of the ClientApp and submitting the required registration data through the /api/v1/authentication/login interface. The request is forwarded

by the API Gateway to the SecurityManager, which validates the completeness and correctness of the submitted information, including password security requirements. [...] If the verification link expires before being used, the SecurityManager invalidates the link and removes the previously stored unverified account data from the SecurityDBMS, forcing the User to repeat the registration process."

* **Output**:

```
title SD1. Sign Up

actor User
participant "Client App" as App
participant "API Gateway" as GW
participant "Security Manager" as SM
participant "Profile Manager" as PM
participant "Mailing Service" as MS
database "Security DBMS" as SDB
database "Profiles DBMS" as PDB

== Registration Request ==
User->App: Click "Sign Up"
activate App
App->User: Show registration form
deactivate App
User->App: Insert email, password...
... [Rest of the generated code] ...
```

– **Text Refinement**

* **Input**: "Rewrite the following paragraph to be more formal and suitable for a Software Engineering Design Document: [Draft text in Italian/English]"

* **Output**: " [Refined and polished English text] "

– **Image Generation**

* **Input**: "Generate a flat, modern icon representing a 'Trip Contribution Service' for a software architecture diagram."

* **Output**: [Generated Image File]

List of Figures

2.1 High-level overview of the BBP system architecture. 7
2.2 Component diagram of the BBP system. 8
2.3 UML Deployment Diagram 14
2.4 Sign Up 21
2.5 Sign In 23
2.6 Delete Account 25
2.7 Logout 27
2.8 Edit Profile 29
2.9 View Trip History 31
2.10 Publish Contribution 33
2.11 View Map 35
2.12 Search Path 37
2.13 Manual Insertion 39
2.14 Automatic Insertion 41
2.15 Enrich Trip with Weather Information 43
2.16 Edit Trip 45

3.1 Sign in 49
3.2 Sign up 49
3.3 Map 50
3.4 Navigation 50
3.5 Profile 51
3.6 Activity 51
3.7 Settings 51
3.8 History 52
3.9 Trip data 52
3.10 Trip recording 53
3.11 modify trip 53
3.12 54
3.13 54

List of Tables

1.1 Acronyms used in the document. 2
1.2 Abbreviations used in the document. 3
1.3 Revision history 3

4.1 Mapping between Functional Requirements and System Components. 59

6.1 Effort spent by each member of the group. 68